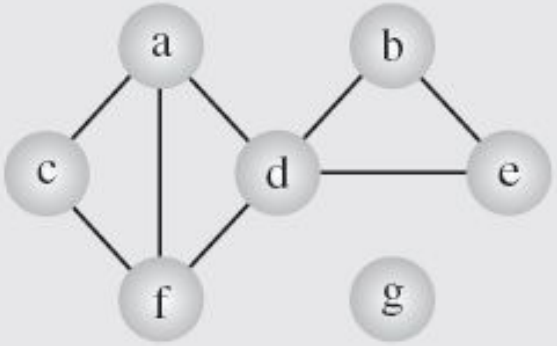
# Graph Representation

- There are two ways of representing graphs:

1. Adjacency matrix
2. Adjacency list

# Adjacency List

- A simple representation is given by an *adjacency list,* which specifies all vertices adjacent to each vertex of the graph.
- Storage space O(|V|+|E|)




(a)

| | | | | |
|---|---|---|---|---|
| **a** | c | d | f | |
| **b** | d | e | | |
| **c** | a | f | | |
| **d** | a | b | e | f |
| **e** | b | d | | |
| **f** | a | c | d | |
| **g** | | | | |

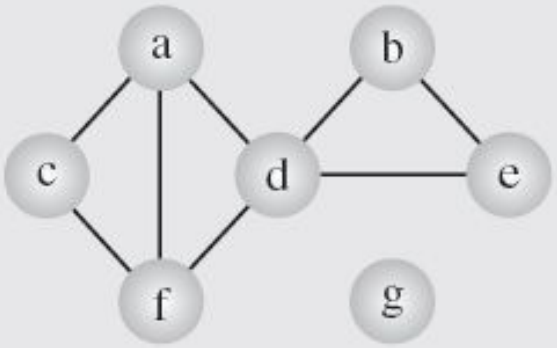
(b)

# Adjacency Matrix

- An *adjacency matrix* of graph G = (V,E) is a binary |V| × |V| matrix such that each entry of this matrix.

$$a_{ij} = \begin{cases} 1 & \text{if there exists an edge}(v_i v_j) \\ 0 & \text{otherwise} \end{cases}$$

Note:Storage space O(|V|x |V|)

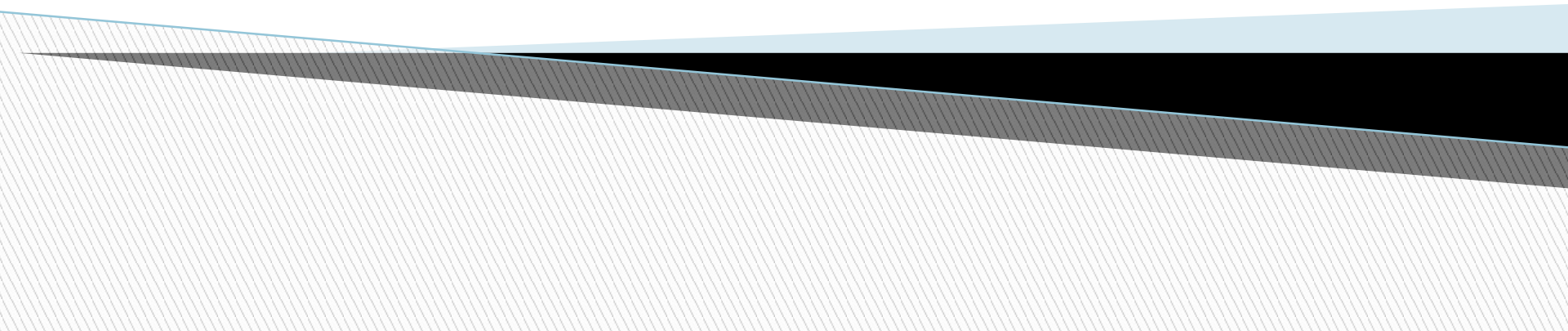



(a)

| | a | b | c | d | e | f | g |
|---|---|---|---|---|---|---|---|
| a | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| b | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| c | 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| d | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| e | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| f | 1 | 0 | 1 | 1 | 0 | 0 | 0 |
| g | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

(d)

# Graph Traversal



# Shortest Path problem

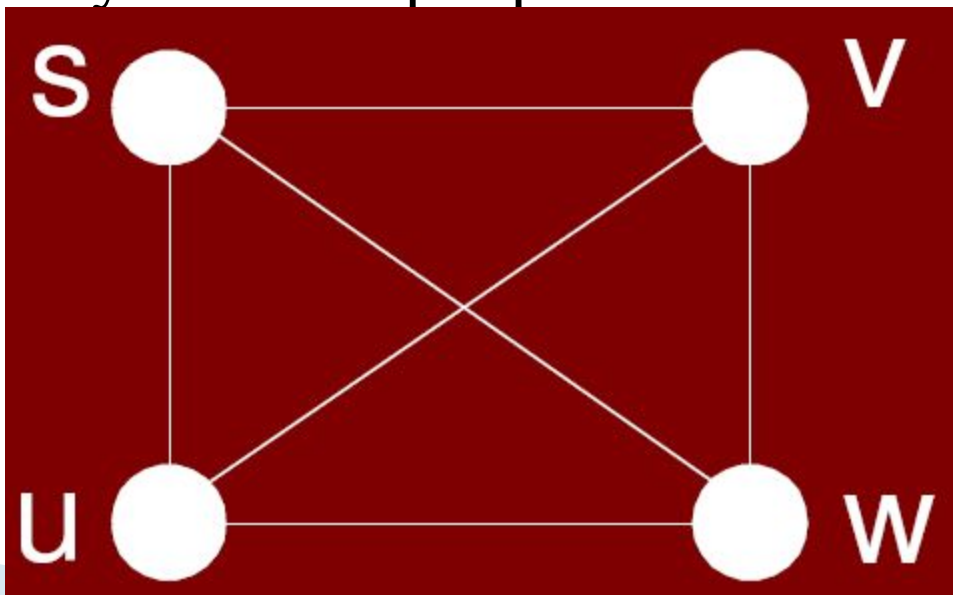
- We are given an undirected graph G = (V, E) and a *source vertex s ϵ V*.
- The *length* of a path in a graph is the number of edges on the path.
- We would like to find the shortest path from s to each other vertex in the graph.
- At here shortest path mean to say to reduce the number of edge in this path.

# Continue….

- **Final Result**
  - The final result will be represented in the following way.
- For each vertex v ϵ V, we will store d[v] which is the *distance* (length of the shortest path) from s to v.
- d[v] will contain the count of minimum number of edges.
- Note that d[s]=0  // source to source
- We will also store a predecessor (or parent) pointer π[v] which is the first vertex along the shortest path if we walk from v backwards to s. We will set π[s] = Nil.

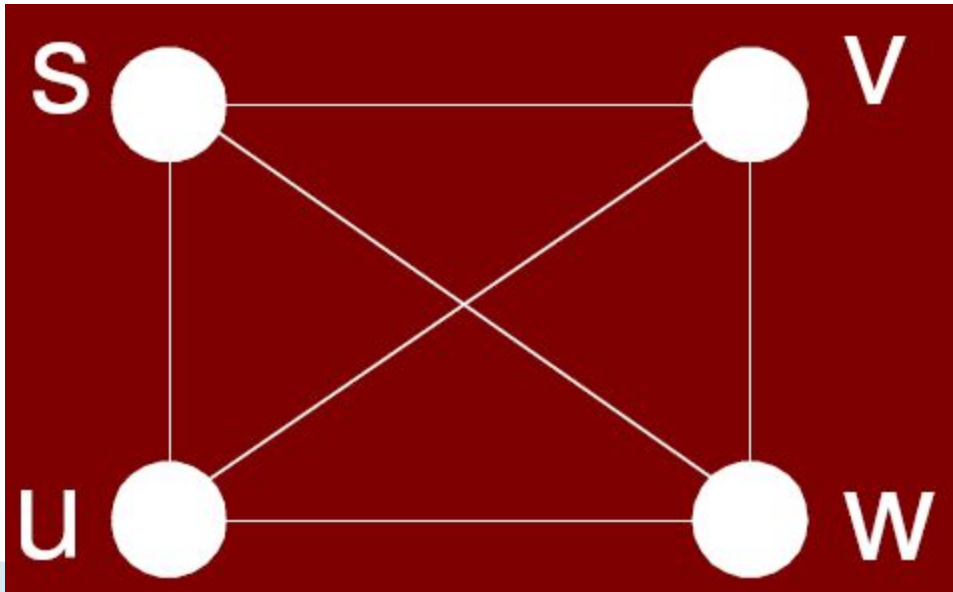
# Simple brute-force strategy

- There is a simple brute-force strategy for computing shortest paths.
- We could simply start enumerating all simple paths starting at s, and keep track of the shortest path arriving at each vertex.
- However, there can be as many as n! simple paths in a graph.
- Clearly this is not feasible



# Simple Brute-Force strategy

- There can be n choices for source node s in our example it is 4.
- There can be n-1 choices for destination node in our example it is 3
- For first hop selection(let s□w can be v, u.) n-2 choices. At here 2
- for 2nd hop n-3(at u can be v and at v can be u) at here 1
- For 3rd hop n-4
- So n=4x3x2x1=24



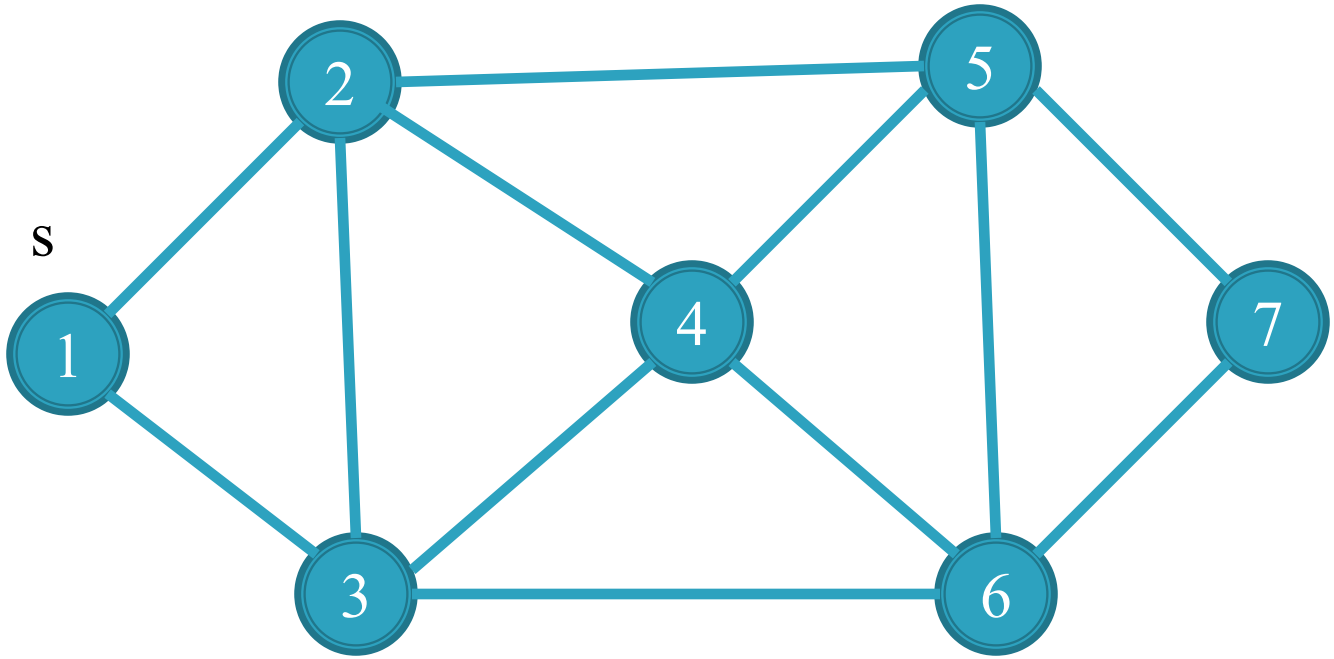
# Efficient Solution for Shortest Path

- Here is a more efficient algorithm for shortest path called the breadth-first search (BFS)
- BFS and DFS(depth-first search ) two traversal algorithms used for graph.
- The simple traversal algorithms used for trees cannot be applied here because graphs may include cycles; hence, the tree traversal algorithms would result in infinite loops.
- First we will have a look on these two algorithm then we will use it in some problems.

# Depth first search (DFS)

```
1.  Traverse(s)
2.      Push (null , s) on stack
3.      While stack is not empty
4.          pop (p , v) from stack
5.              if (v is unmarked)
6.                  then mark v
7.                  parent(v)=p
8.                  for each edge (v , w)
9.                      push (v , w) on stack
```

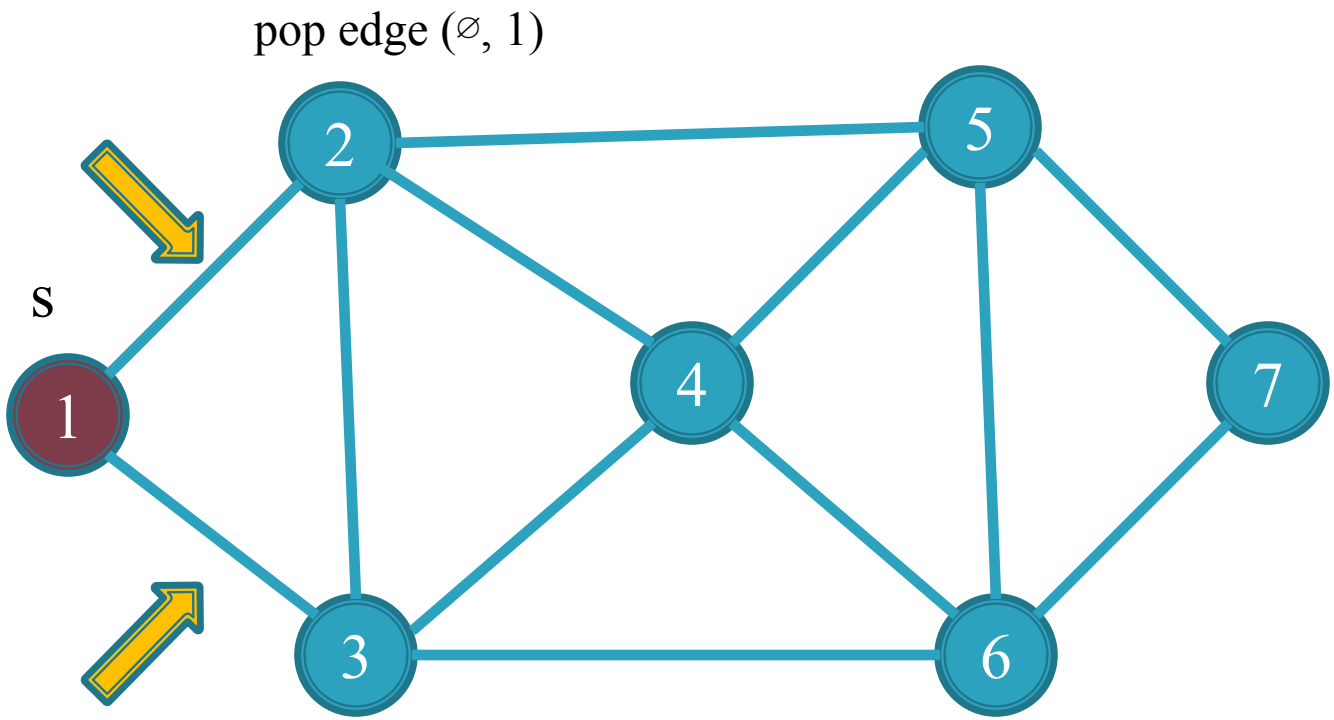
# Trace of Depth-first-search algorithm:



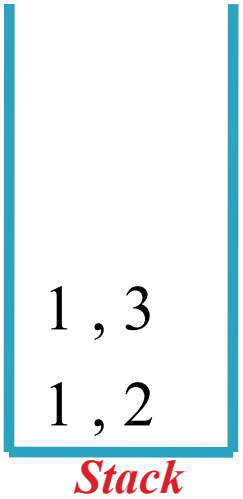

∅, 1

*Stack*

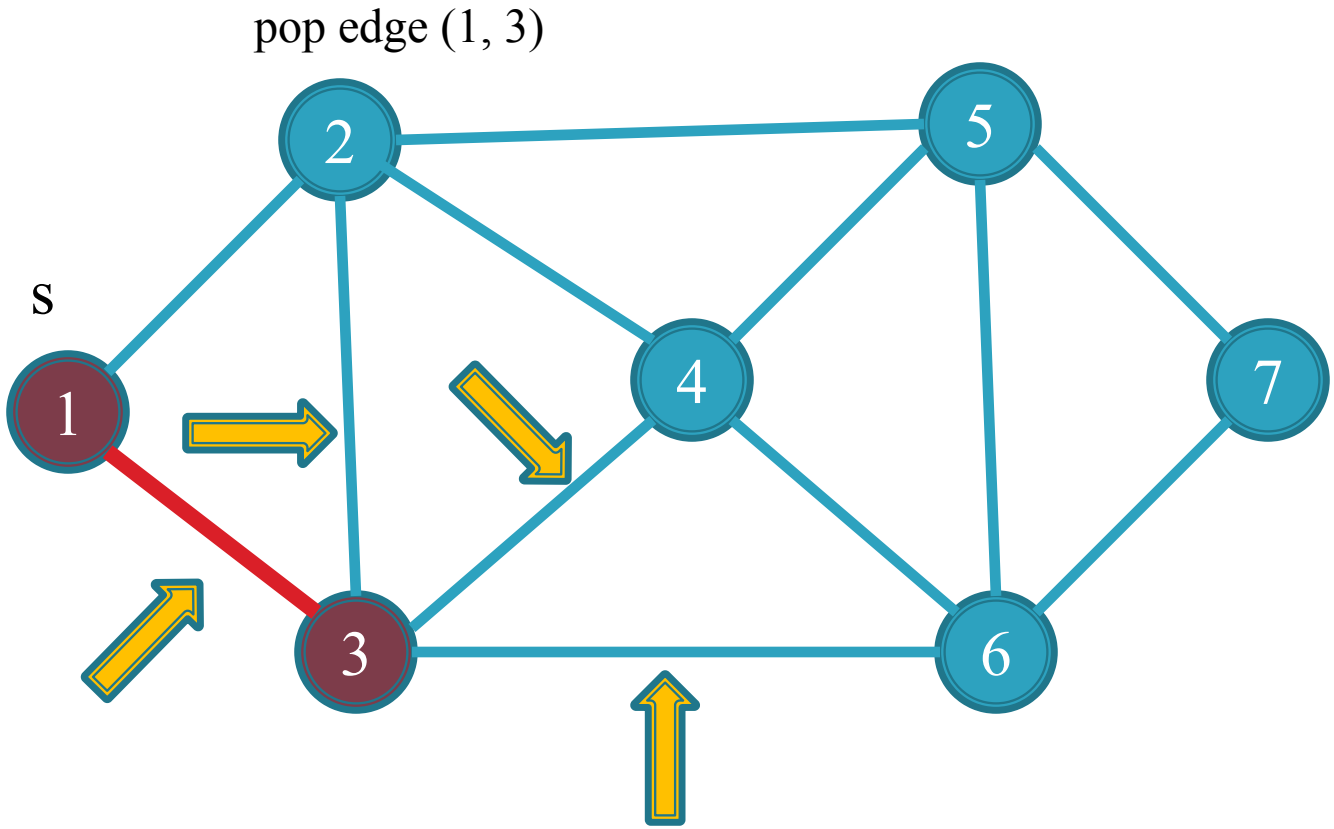
# Trace of Depth-first-search algorithm:




Note: no need to mark edge at here because parent node is NULL



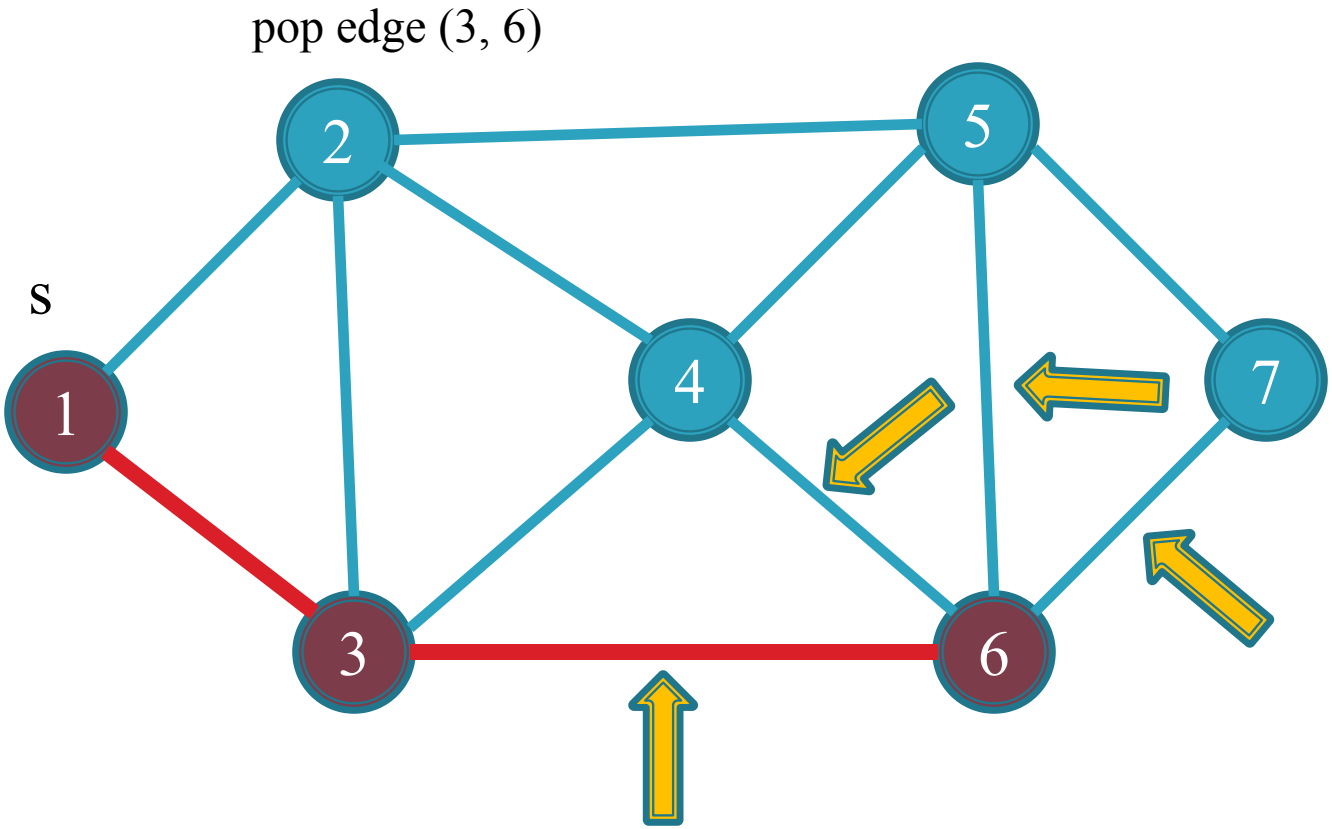
# Trace of Depth-first-search algorithm:




3 , 6
3 , 1
3 , 4
3 , 2
1 , 2

*Stack*

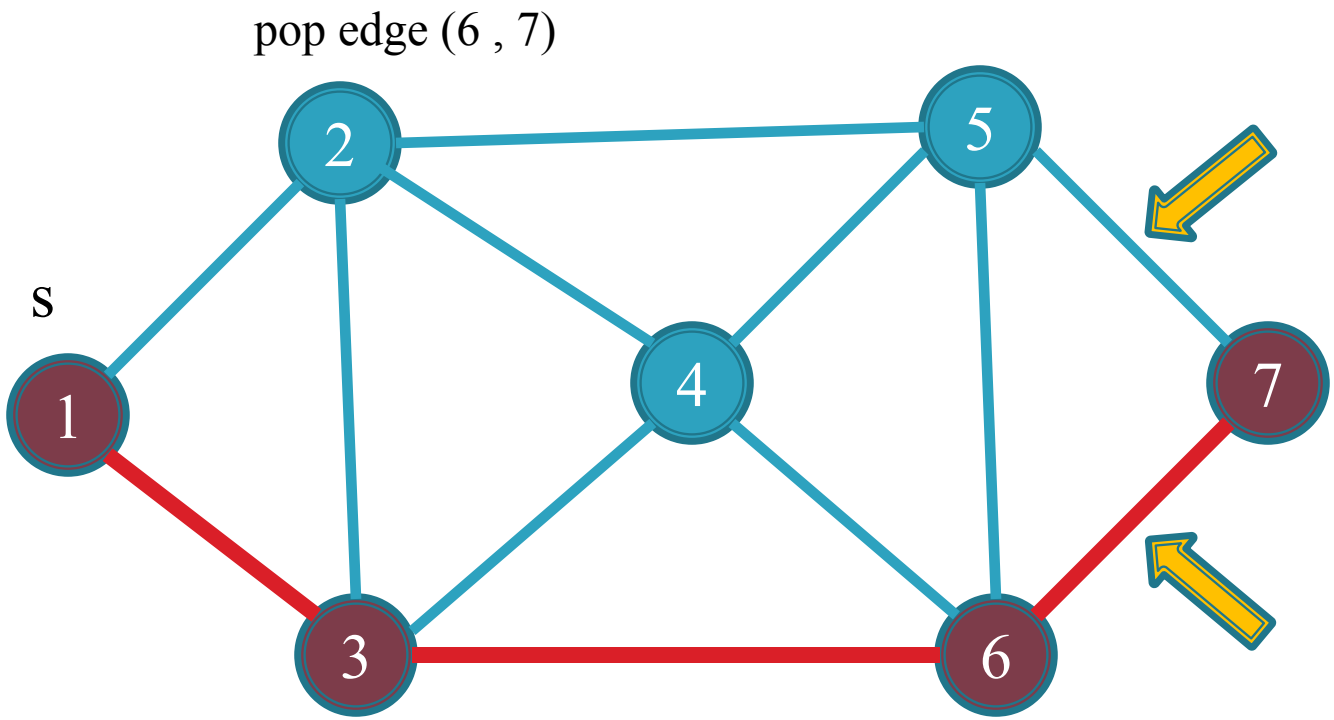
# Trace of Depth-first-search algorithm:




6 , 7
6 , 3
6 , 4
6 , 5
3 , 1
3 , 4
3 , 2
1 , 2

*Stack*

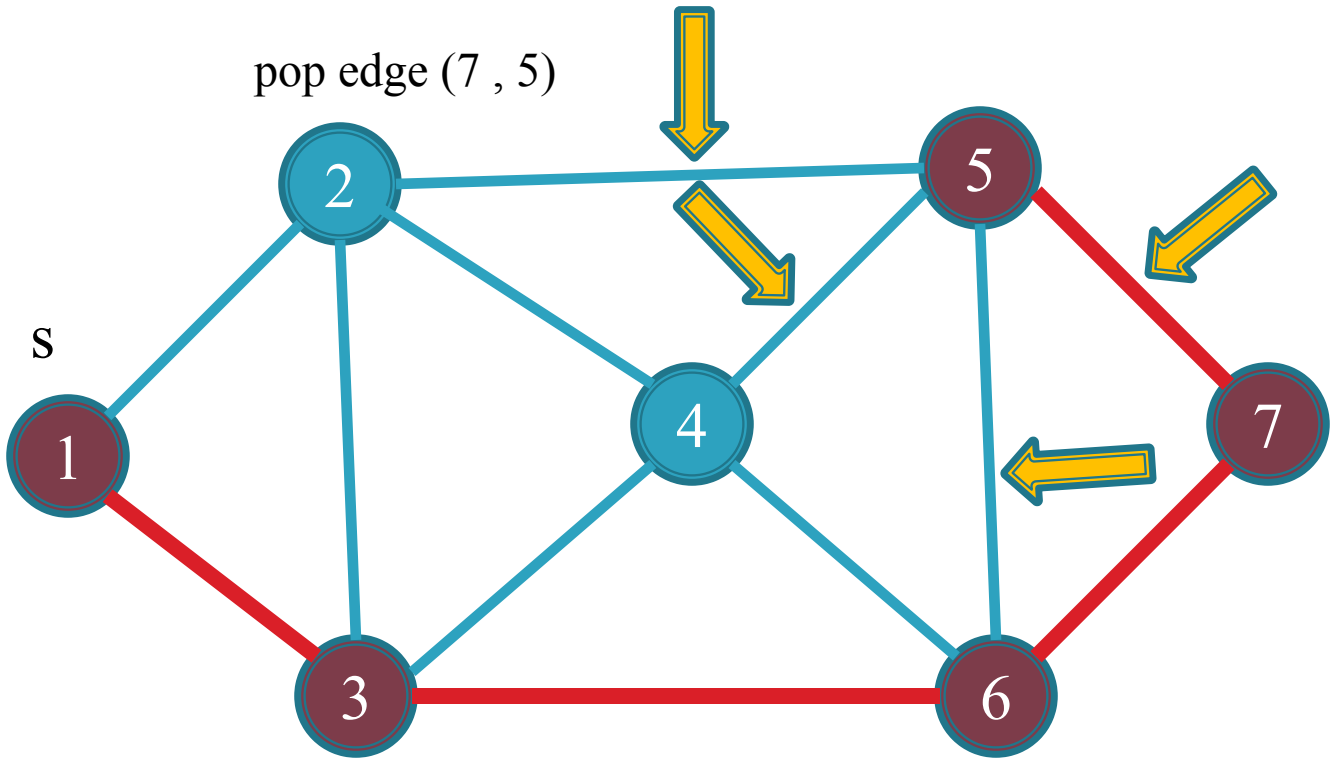
# Trace of Depth-first-search algorithm:




7 , 5
7 , 6
6 , 3
6 , 4
6 , 5
3 , 1
3 , 4
3 , 2
1 , 2

*Stack*

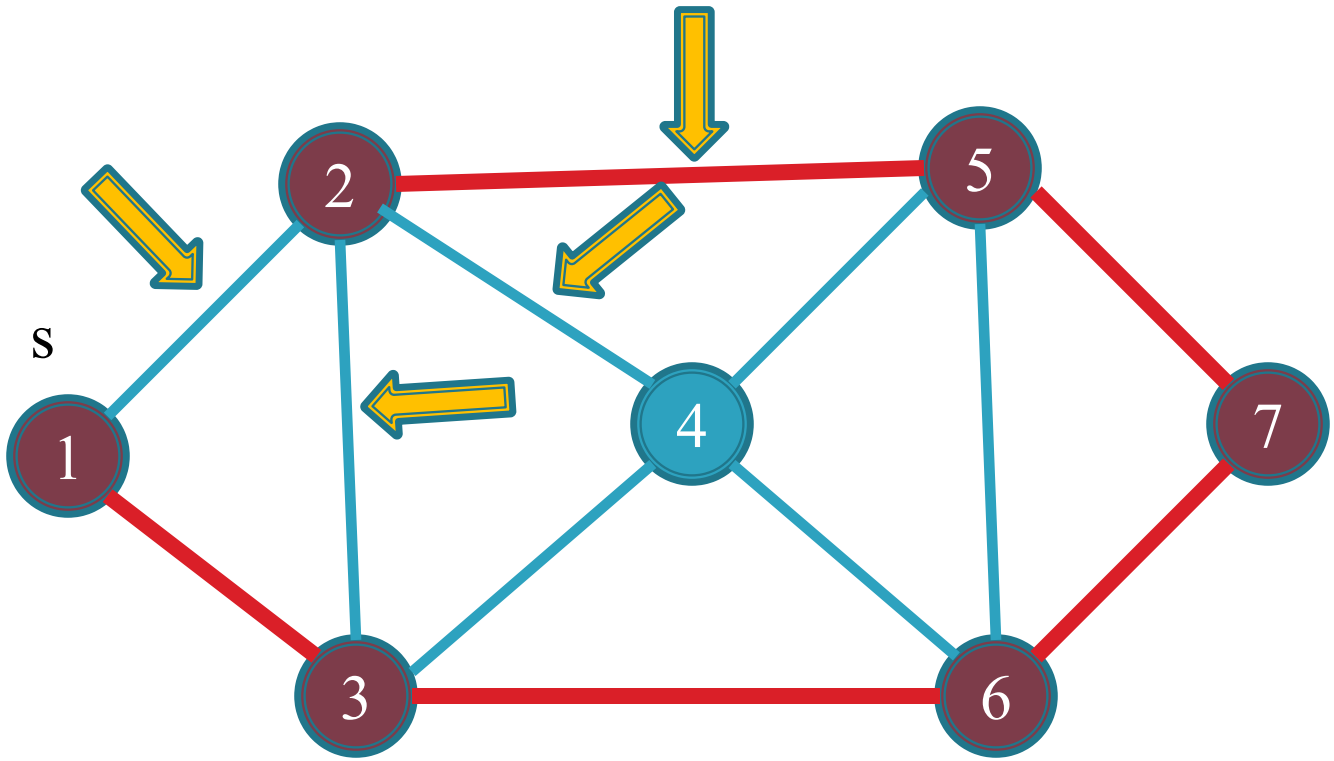
# Trace of Depth-first-search algorithm:




5 , 2
5 , 7
5 , 4
5 , 6
7 , 6
6 , 3
6 , 4
6 , 5
3 , 1
3 , 4
3 , 2
1 , 2

*Stack*

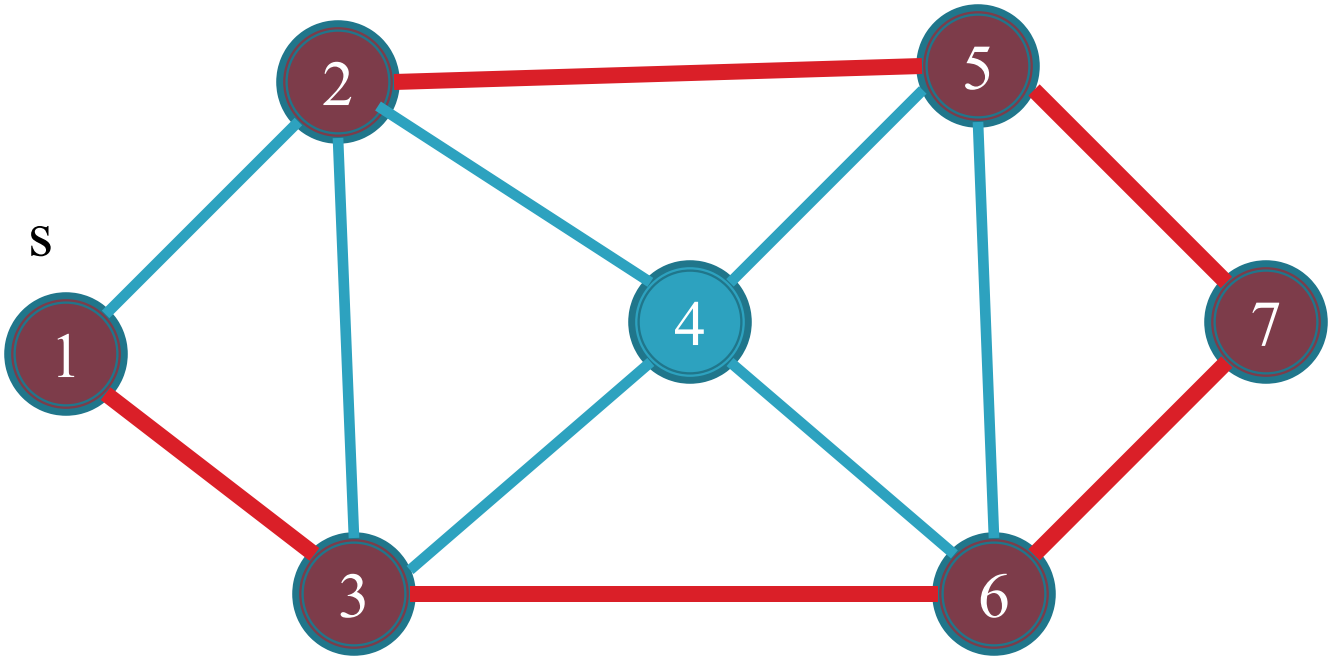
Pop edge (5 , 2)




2 , 1
2 , 5
2 , 3
2 , 4
5 , 7
5 , 4
5 , 6
7 , 6
6 , 3
6 , 4
6 , 5
3 , 1
3 , 4
3 , 2
1 , 2

*Stack*

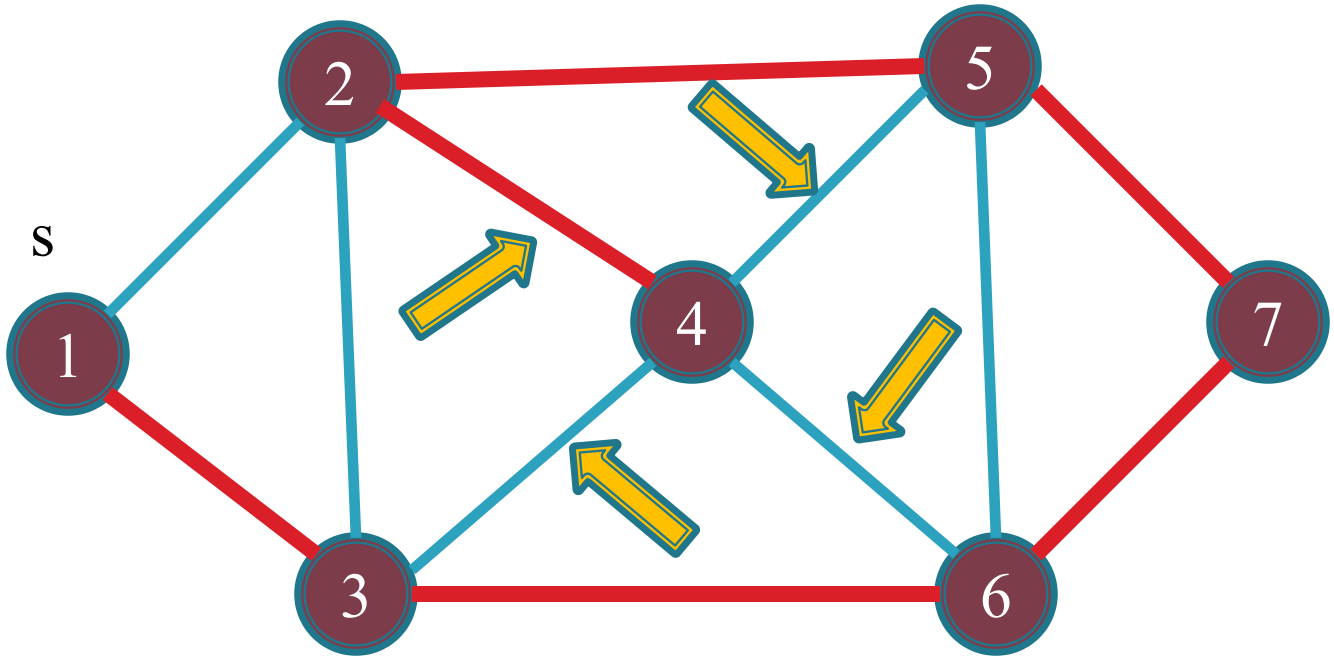
Pop edge (2,1) vertex 1 is already marked, pseudo code line no. 5 if condition fail.
Similarly edge (2,5) and (2,3)




2 , 4
5 , 7
5 , 4
5 , 6
7 , 6
6 , 3
6 , 4
6 , 5
3 , 1
3 , 4
3 , 2
1 , 2

*Stack*

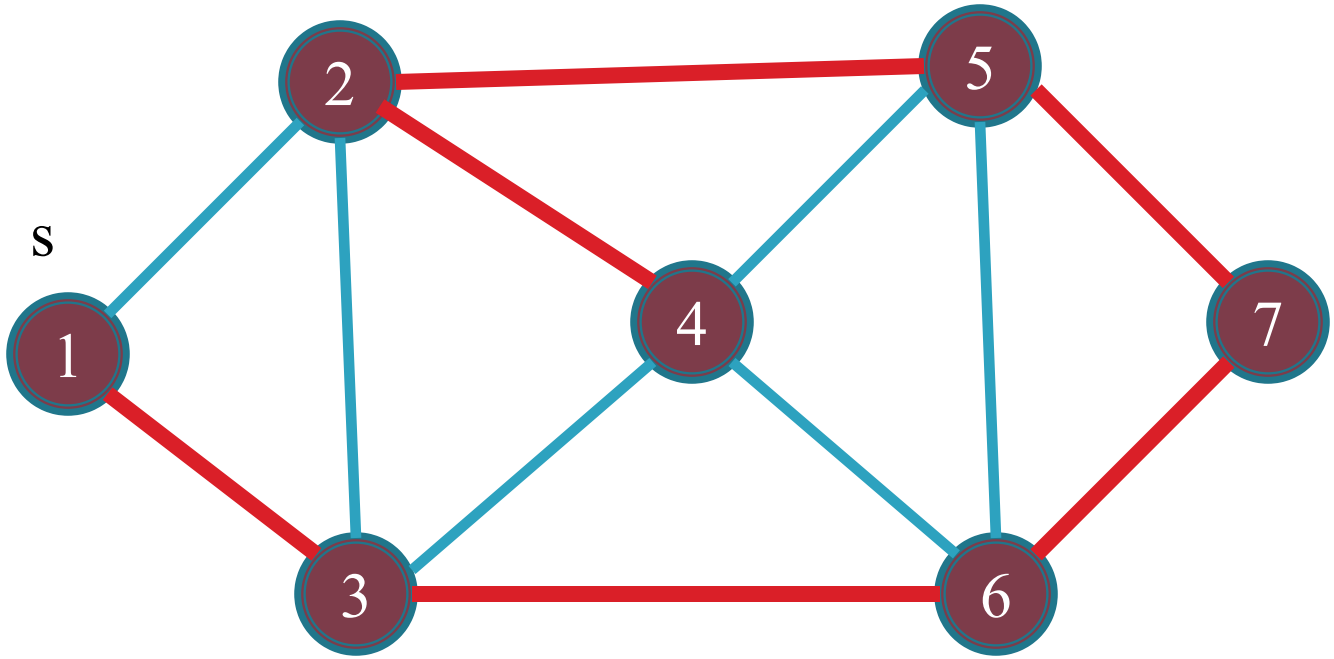
pop edge (2 , 4)




4 , 6
4 , 2
4 , 3
4 , 5
5 , 7
5 , 4
5 , 6
7 , 6
6 , 3
6 , 4
6 , 5
3 , 1
3 , 4
3 , 2
1 , 2

*Stack*

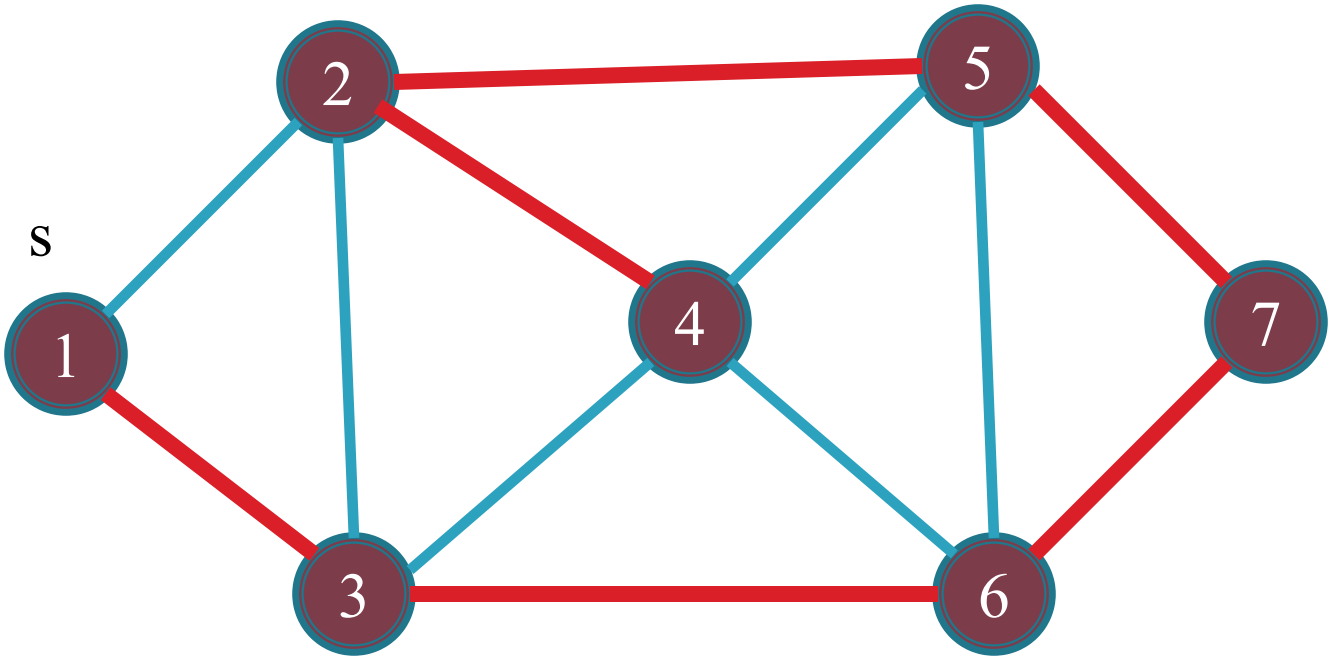
Pop edge (4,6) vertex 6 is already marked, pseudo code line no. 5 if condition fail.
Similarly edge (4,2) , (4,3) and (4,5)




5 , 7
5 , 4
5 , 6
7 , 6
6 , 3
6 , 4
6 , 5
3 , 1
3 , 4
3 , 2
1 , 2

*Stack*

**Similarly all edges remove and stack becomes empty.**




**Note:** these read edges are known as tree edges.

Any edge that is part of a path taken in a graph traversal algorithm is known as a ***tree edge***

Stack

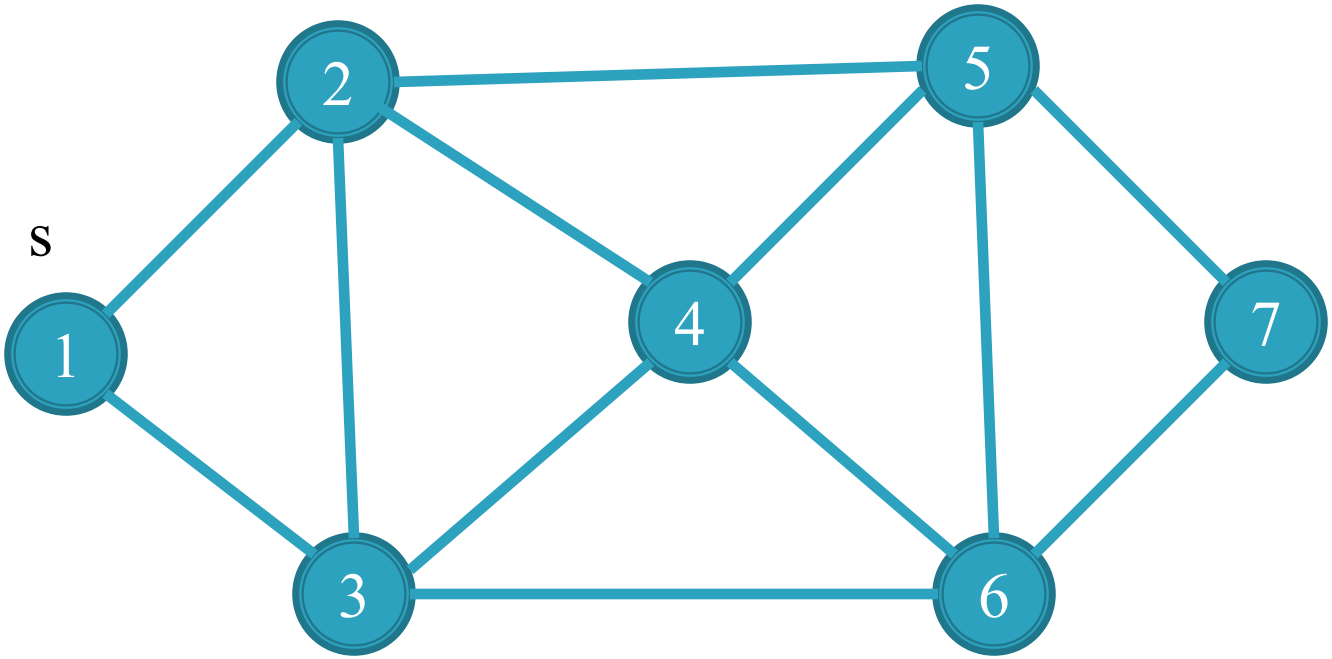
# BFS

- The BFS algorithm traverses the graph in a way that *looks* how a physical wave would move.
- An example is given below.

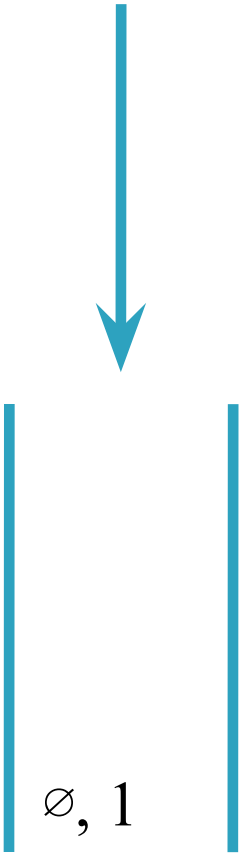
# Breadth First Search (BFS)

```
1.  Traverse(s)
2.      enqueue (null , s) on queue
3.      While queue is not empty
4.          dequeue (p , v) from queue
5.              if (v is unmarked)
6.                  then mark v
7.                  parent(v)=p
8.                  for each edge (v , w)
9.                      enqueue (v , w) on queue
```

# Trace of Breadth-first-search algorithm:

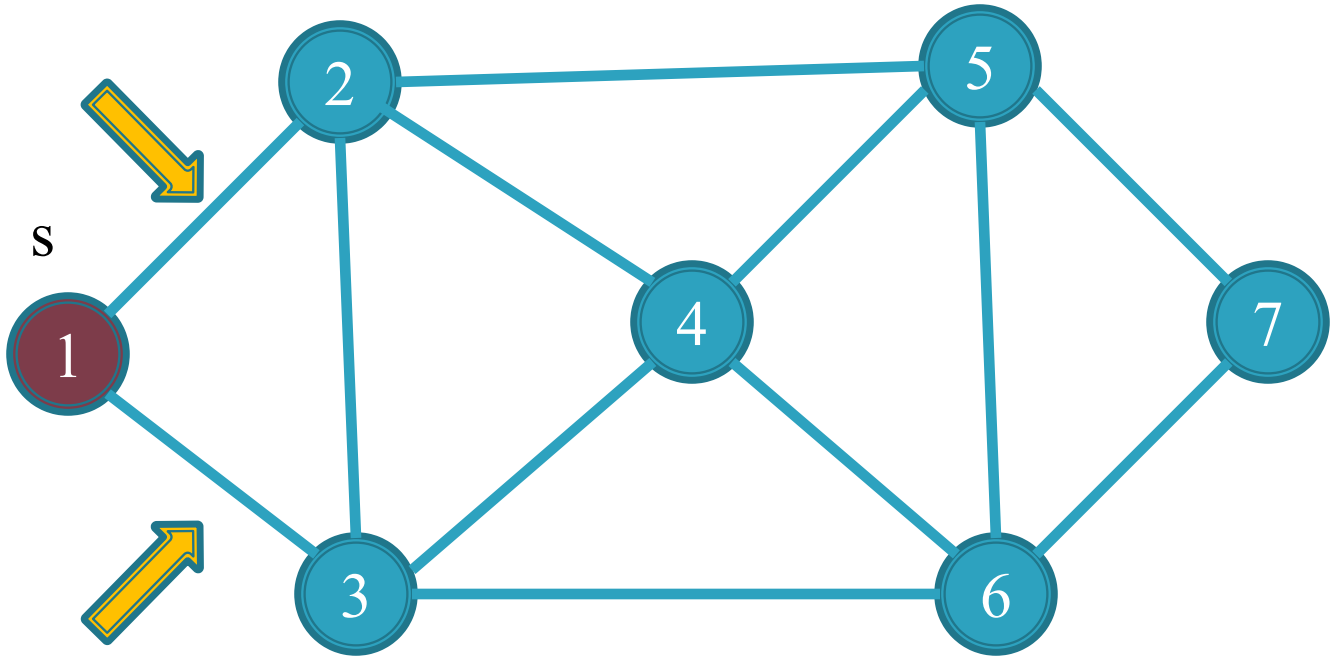



Enqueue from this side



# Trace of Breadth-first-search algorithm:

From edge (∅,1)




Enqueue from this side

1 , 3

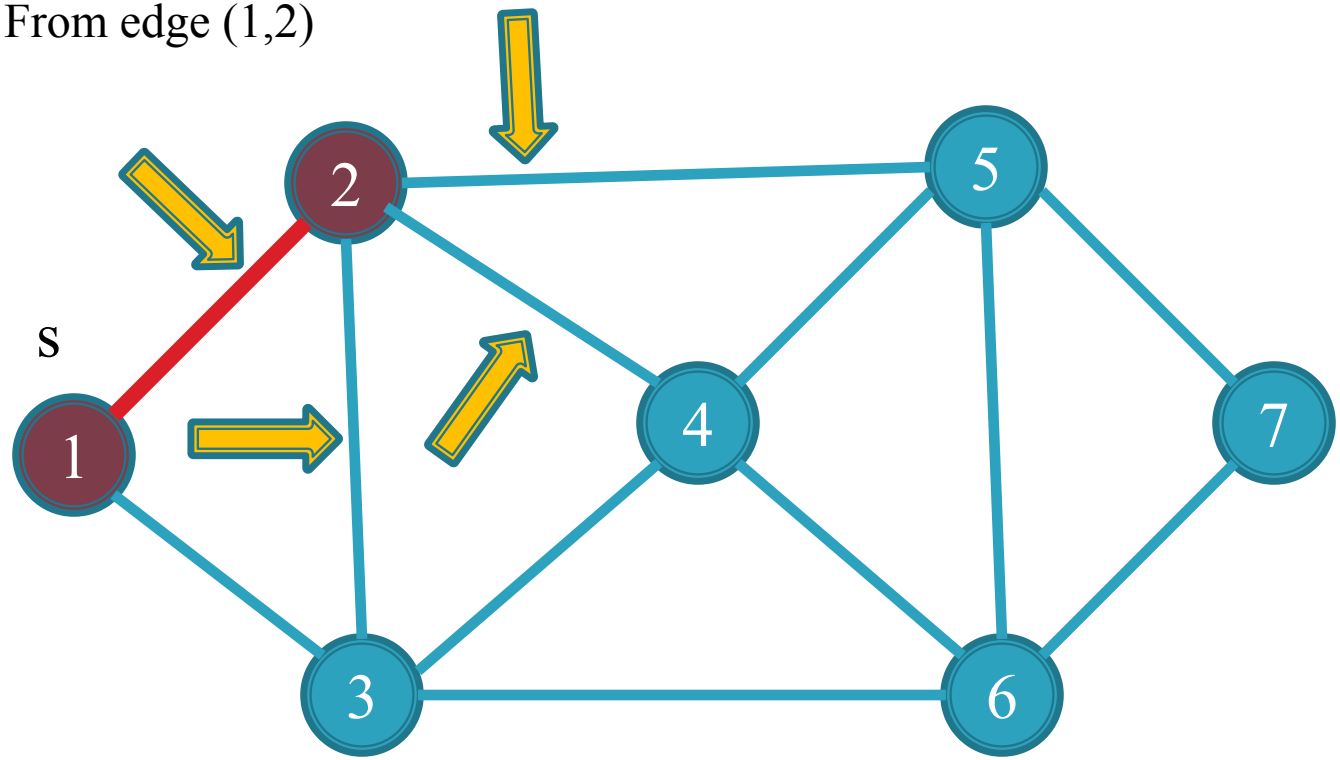
1 , 2

*Queue*

Note: no need to mark edge at here because parent node is NULL

# Trace of Breadth-first-search algorithm:

From edge (1,2)

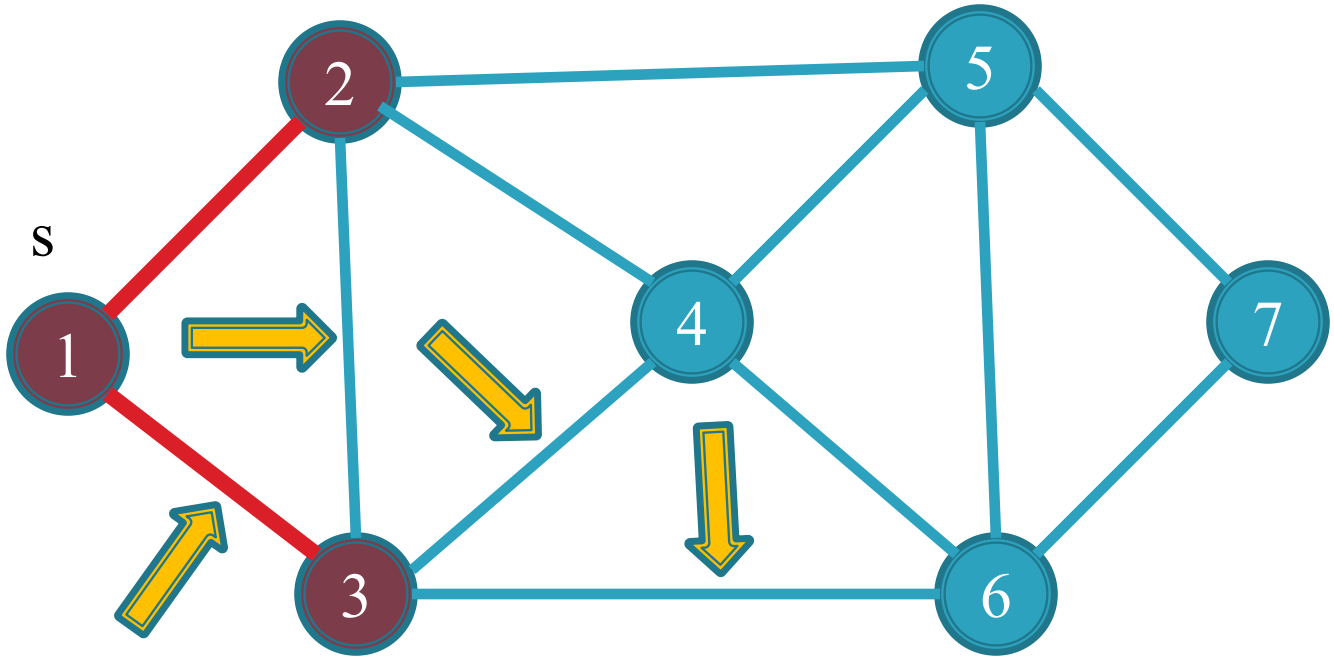



2 , 1
2 , 4
2 , 3
2 , 5
1 , 3

*Queue*

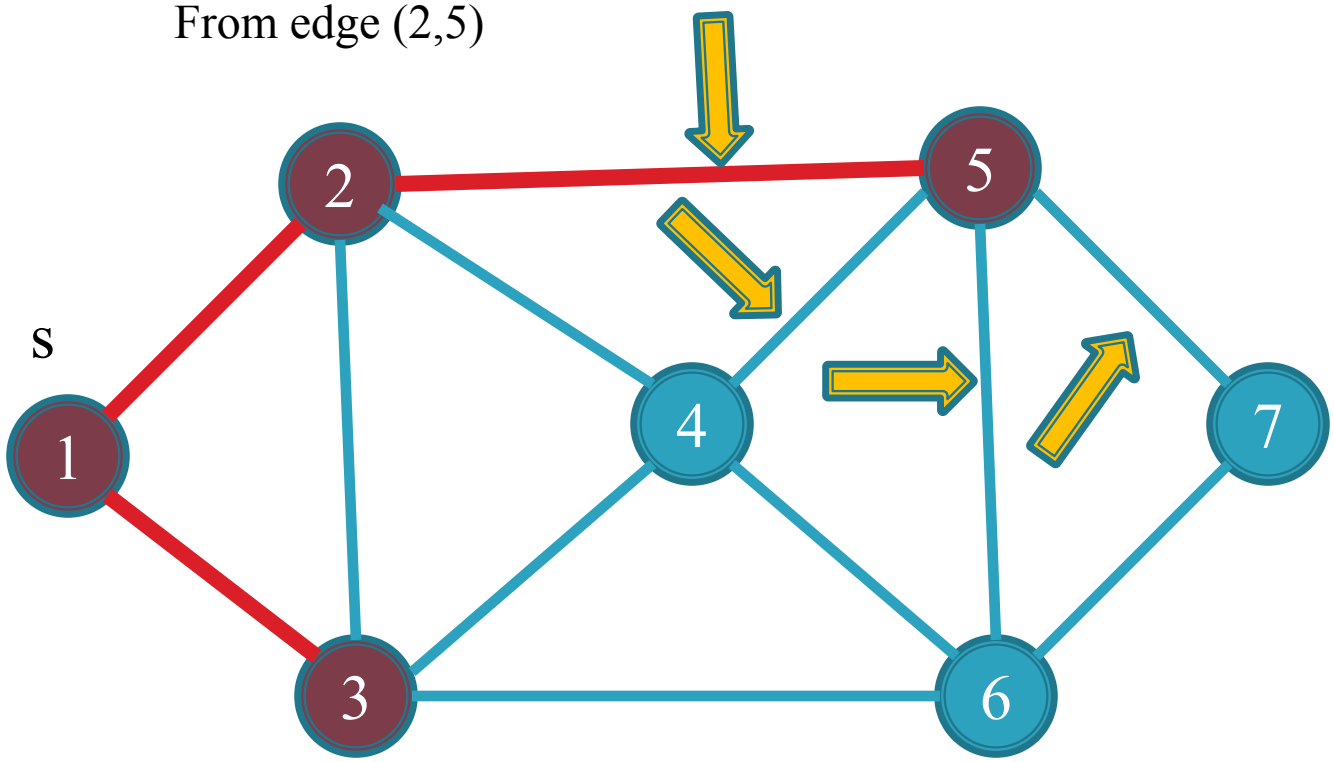
# Trace of Breadth-first-search algorithm:

From edge (1,3)




3 , 6
3 , 4
3 , 2
3 , 1
2 , 1
2 , 4
2 , 3
2 , 5

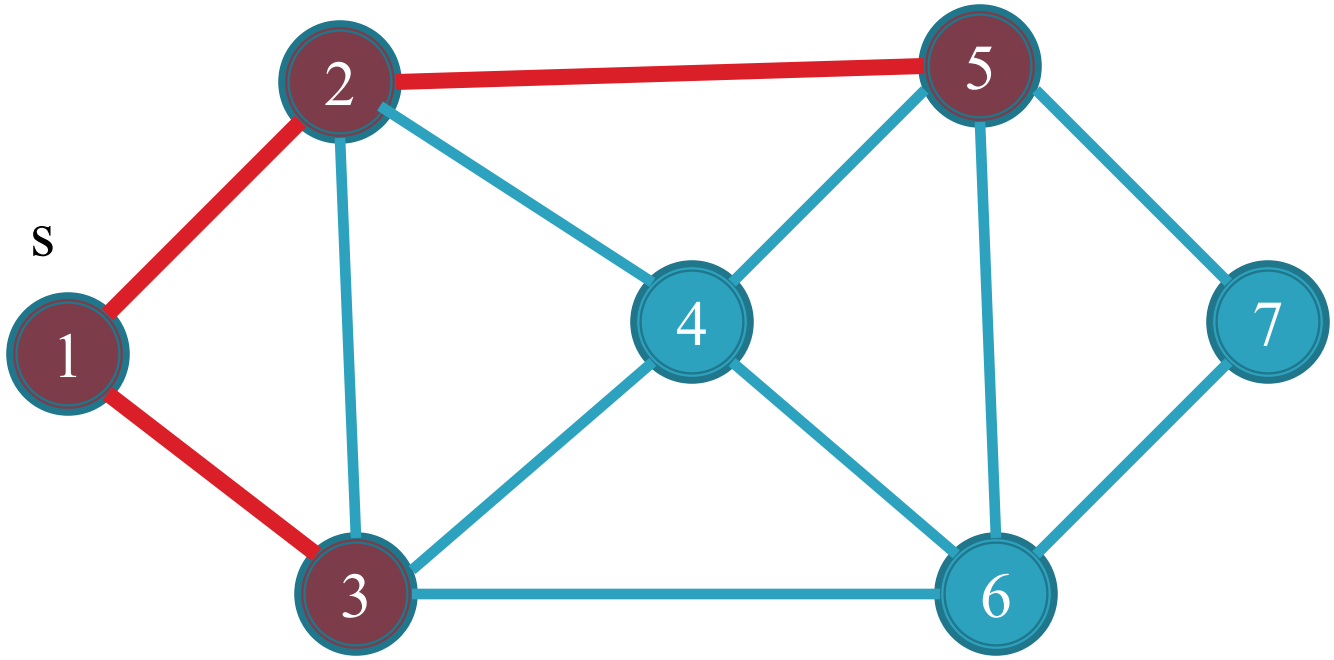
*Queue*




5 , 4
5 , 6
5 , 2
5 , 7
3 , 6
3 , 4
3 , 2
3 , 1
2 , 1
2 , 4
2 , 3

*Queue*

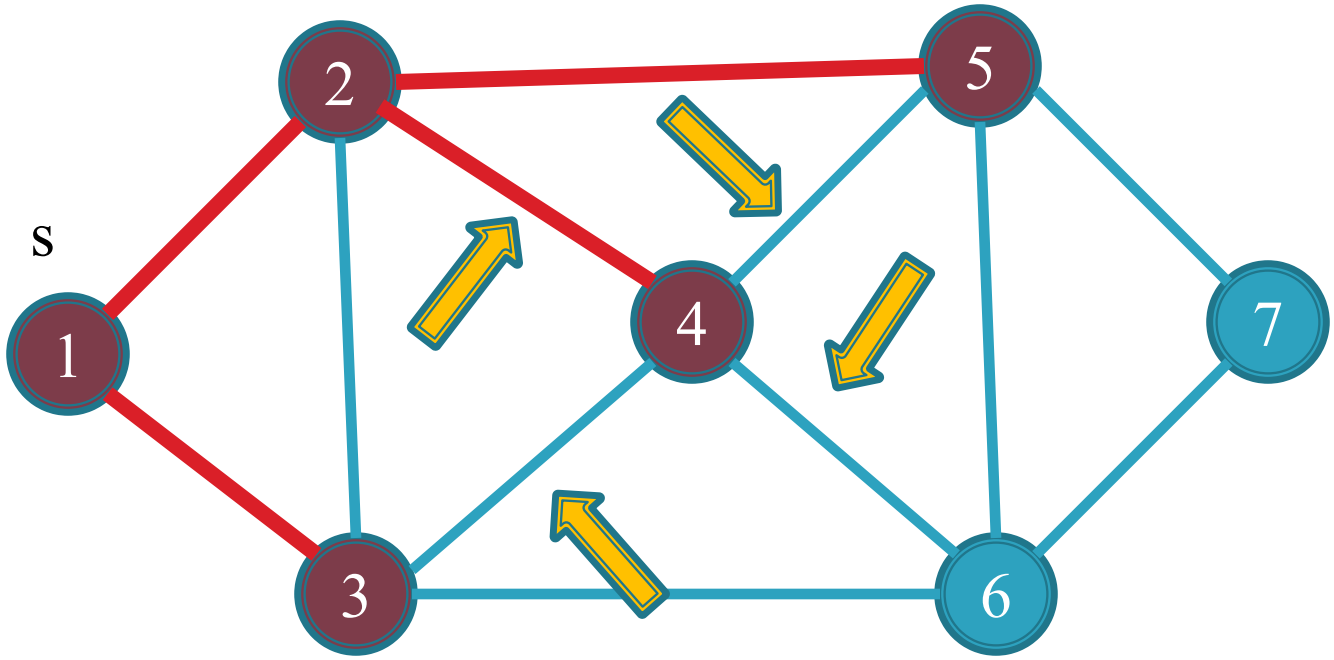
From edge (2,3) the vertex 3 is marked




5 , 4
5 , 6
5 , 2
5 , 7
3 , 6
3 , 4
3 , 2
3 , 1
2 , 1
2 , 4

*Queue*

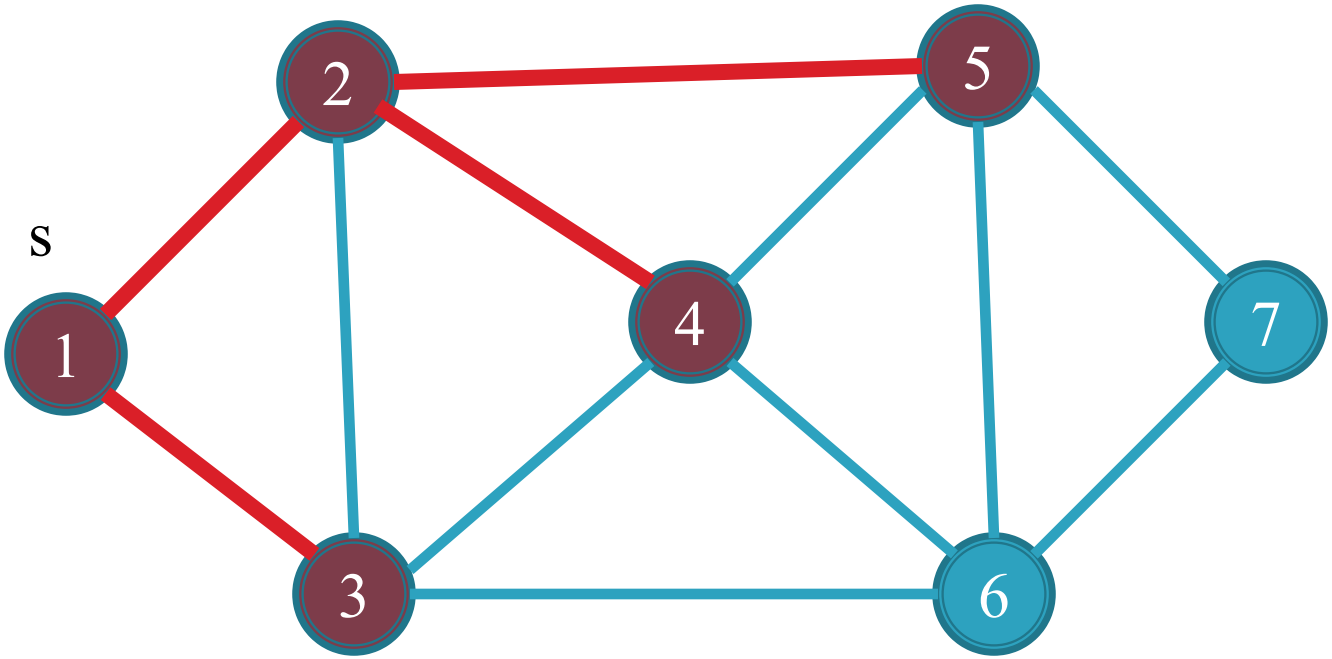
From edge (2,4)




4 , 2
4 , 6
4 , 5
4 , 3
5 , 4
5 , 6
5 , 2
5 , 7
3 , 6
3 , 4
3 , 2
3 , 1
2 , 1

*Queue*

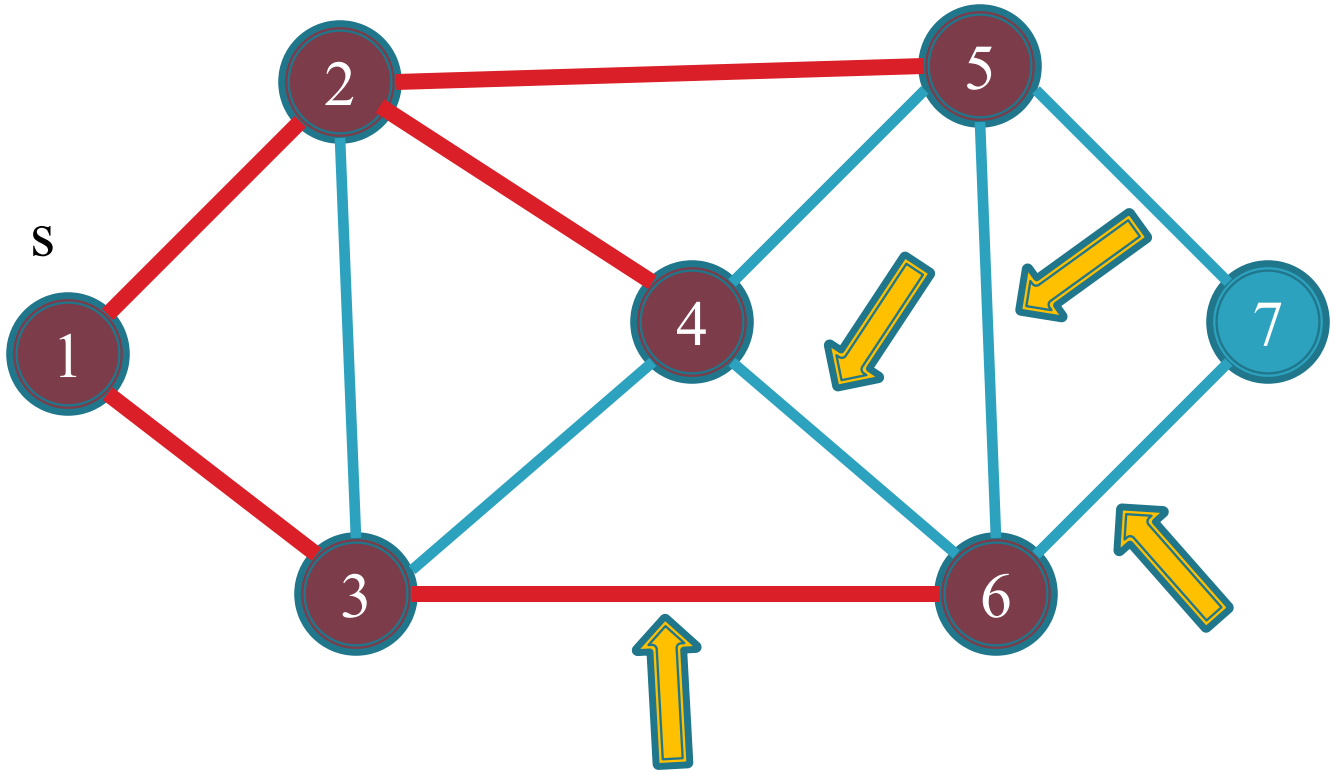
From edge (2,1) vertex 1 is marked
From edge (3,1) vertex 1 is marked
From edge (3,2) vertex 2 is marked
From edge (3,4) vertex 4 is marked




4 , 2
4 , 6
4 , 5
4 , 3
5 , 4
5 , 6
5 , 2
5 , 7
3 , 6

*Queue*

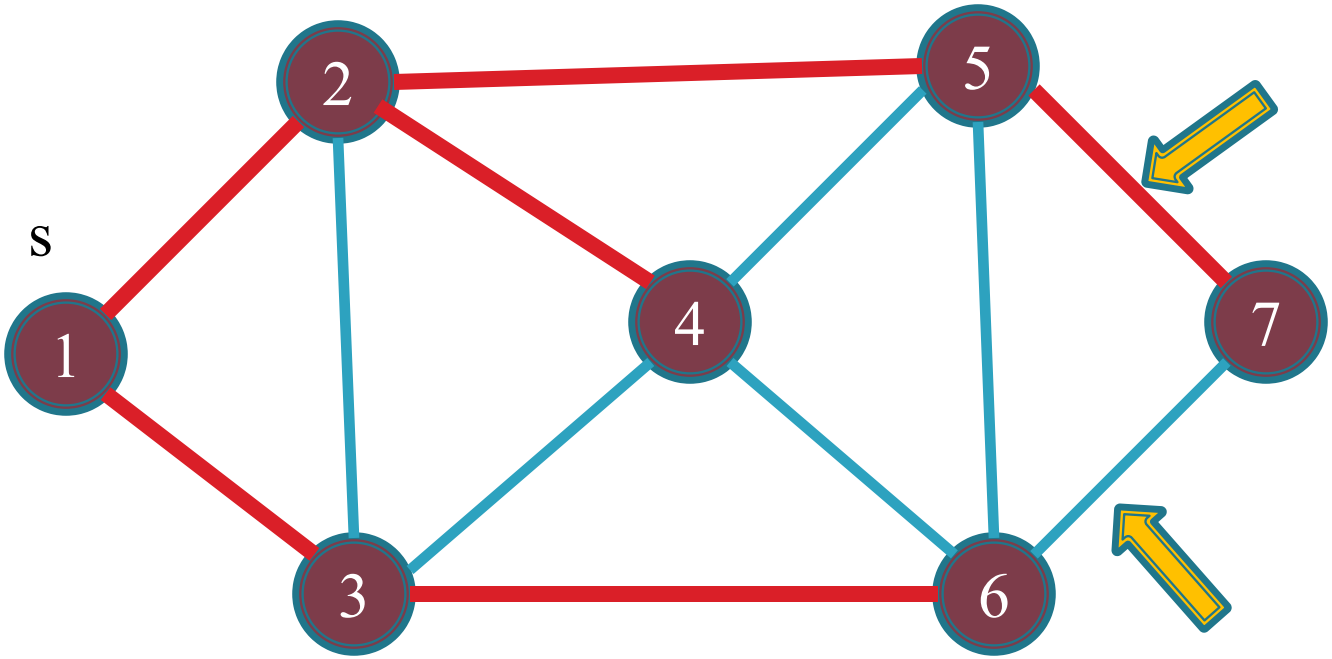
From edge (3,6)




6 , 7
6 , 4
6 , 5
6 , 3
4 , 2
4 , 6
4 , 5
4 , 3
5 , 4
5 , 6
5 , 2
5 , 7

*Queue*

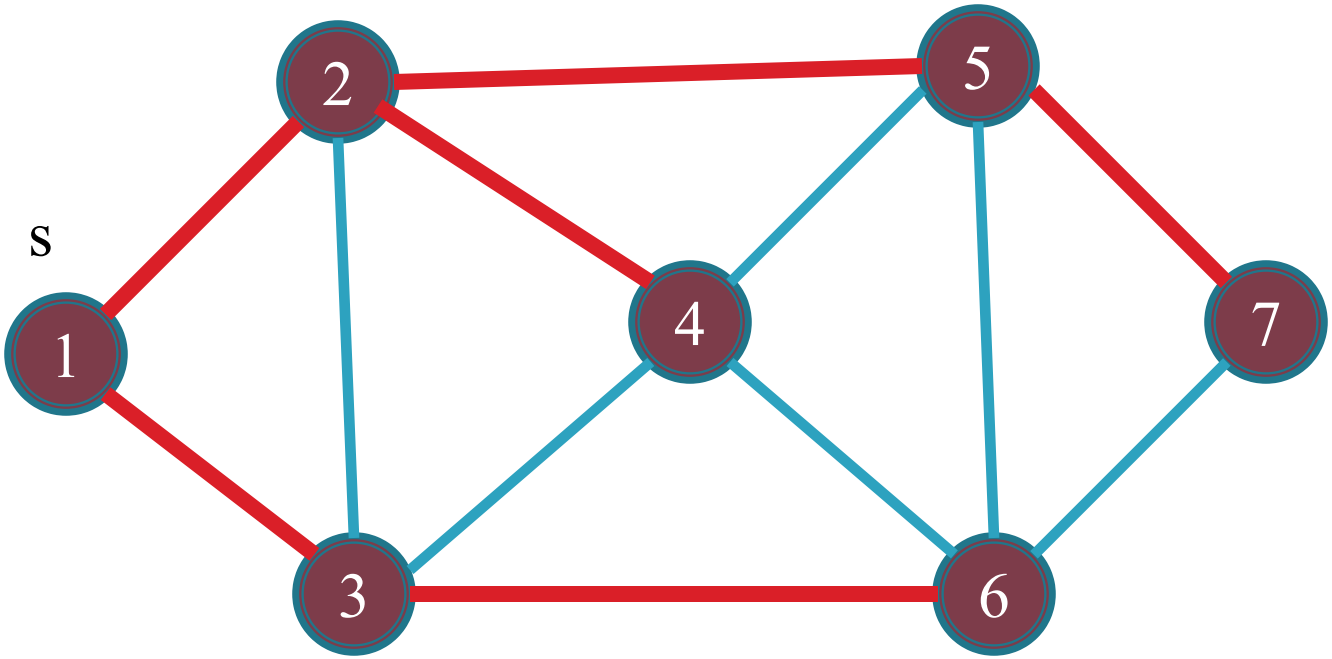
From edge (5,7)




7 , 5
7 , 6
6 , 7
6 , 4
6 , 5
6 , 3
4 , 2
4 , 6
4 , 5
4 , 3
5 , 4
5 , 6
5 , 2

*Queue*

All vertex are marked now queue will become empty



# Time Complexity Analysis

- Line number 3 runs equal to twice of no. of edges i.e $2|E| = O(|E|)$
- Line number 5 runs exactly to no. of vertex i.e. $|V| = O(|V|)$
- Line number 8 and 9 runs $|V|$ plus the number of node adjacent to list which is equal to $2|E|$. So total time of line 8 and 9 is $O(|V|+|E|)$.
- $O(|V|+|E|)$ is dominating factor of these algorithm

# Time Complexity in Adjacency List

- This is linear time algorithm.
- O(|v|+|E|)

- V≤E+1 //at least V=E+1 for connected graph
- So O(|v|+|E|)
- =O(|E|+|E|)
- =O(2|E|)
- =O(|E|)

# Time Complexity in Adjacency Matrix

- The finding of all the neighbors of vertex in line 8 takes O(V) time in adjacency matrix.

- Thus depth-first and breadth-first take $O(V^2)$ time overall.

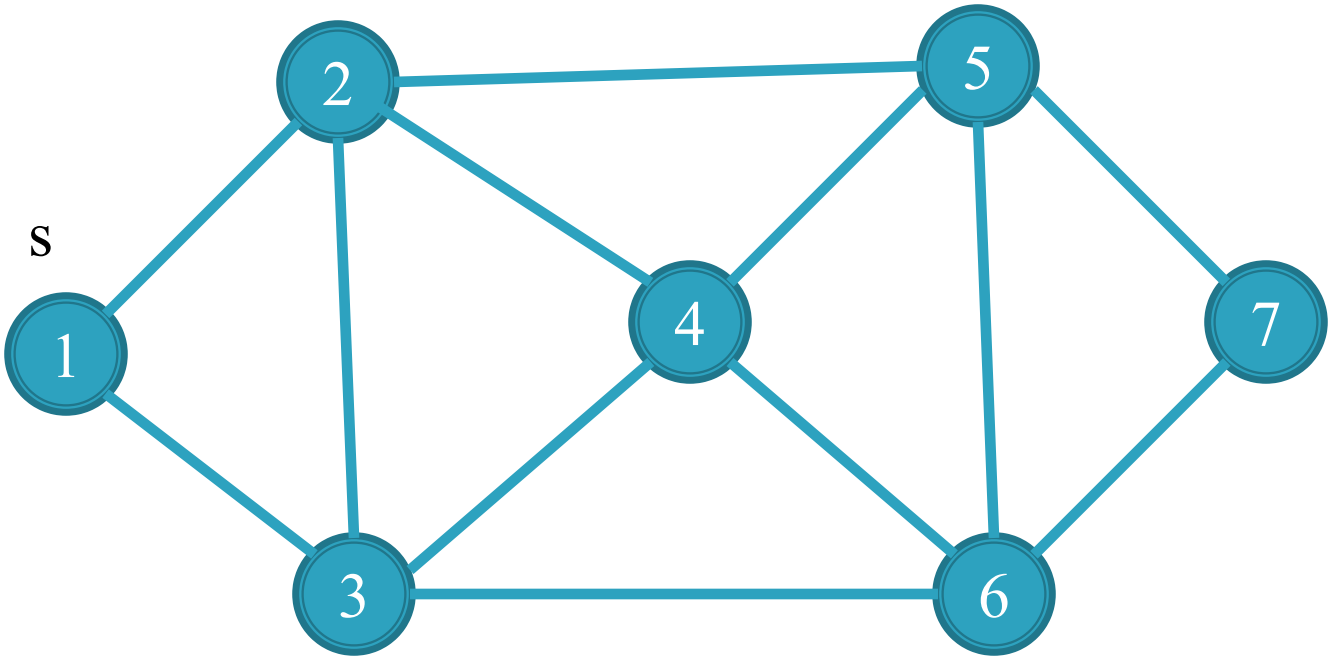
# Find the Connected Graph

- **Problem:** Find the given graph is connect or not?
- **Solution:**
  - First apply any DFS or BFS algorithm on the given graph.
  - When algorithm is completed then check, if all the vertices of the graph are marked then graph is connected otherwise not.

# Shortest path with BFS

```
Traverse(s)
   enqueue (null , s) on queue
   While queue is not empty
    dequeue (p , v) from queue
         if (v is unmarked)
              then mark v
              parent(v)=p
              d[v]=d[p]+1  //d[null]=-1
              for each edge (v , w)
                   enqueue (v , w) on queue
```

# Trace of Breadth-first-search algorithm:




Enqueue from this side

Distance of node is initialize with infinity.

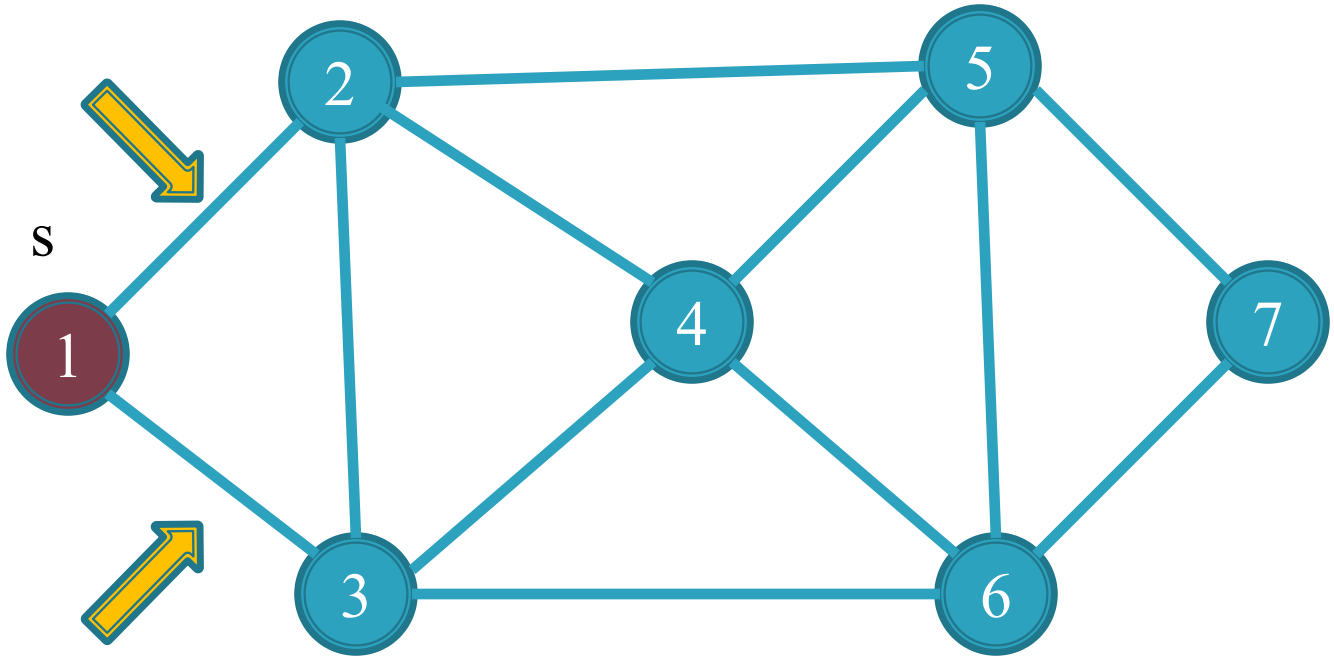
| ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

∅, 1

*Queue*

# Trace of Breadth-first-search algorithm:

From edge (∅,1)




Enqueue from this side

distance from null = -1, so d[1]=-1+1=0

| 0 | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

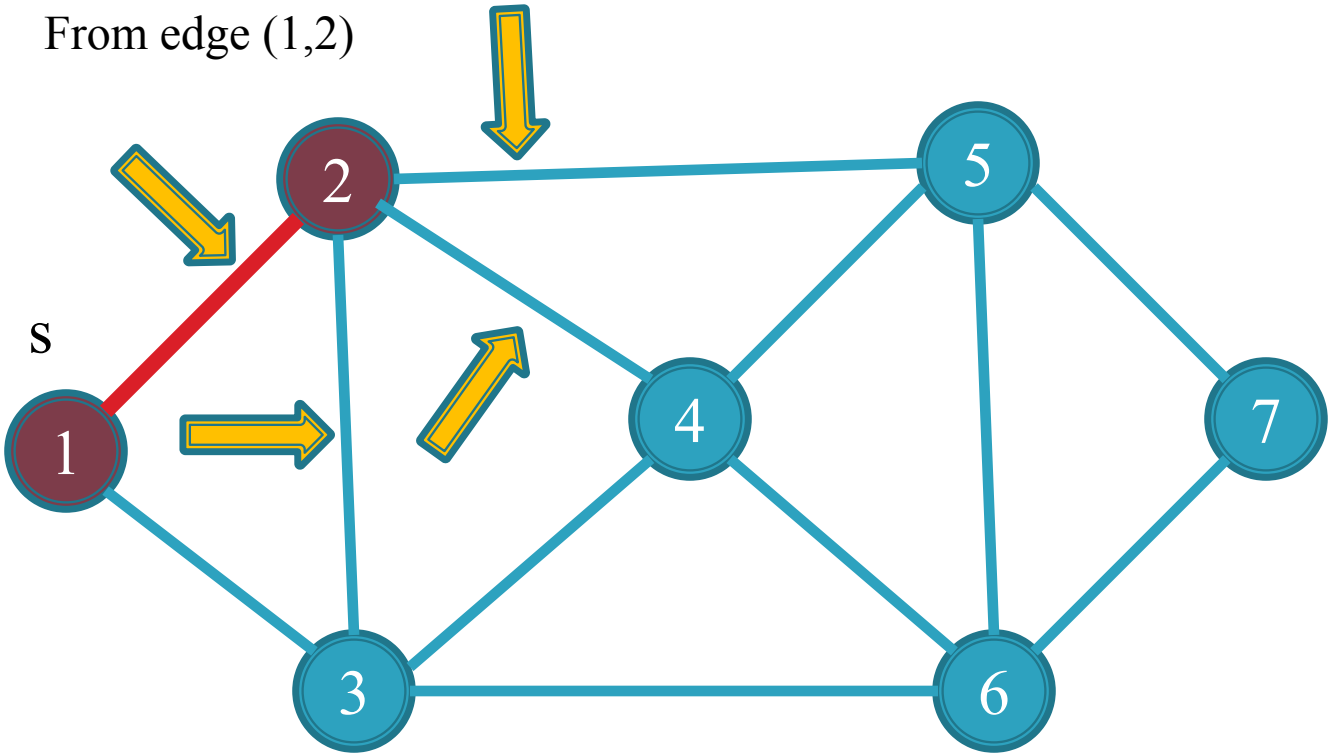
1 , 3

1 , 2

*Queue*

# Trace of Breadth-first-search algorithm:

From edge (1,2)




d[2]=d[1]+1=0+1=1

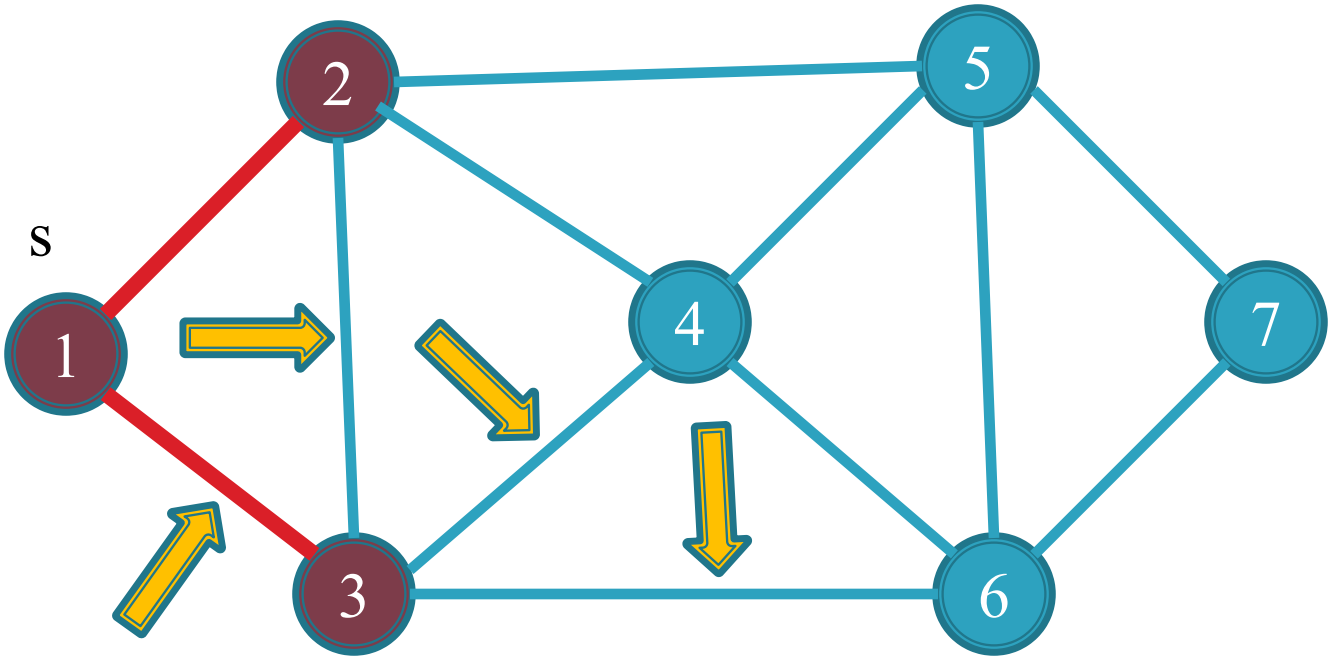
| 0 | 1 | ∞ | ∞ | ∞ | ∞ | ∞ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

Queue

2 , 1
2 , 4
2 , 3
2 , 5
1 , 3

# Trace of Breadth-first-search algorithm:

From edge (1,3)



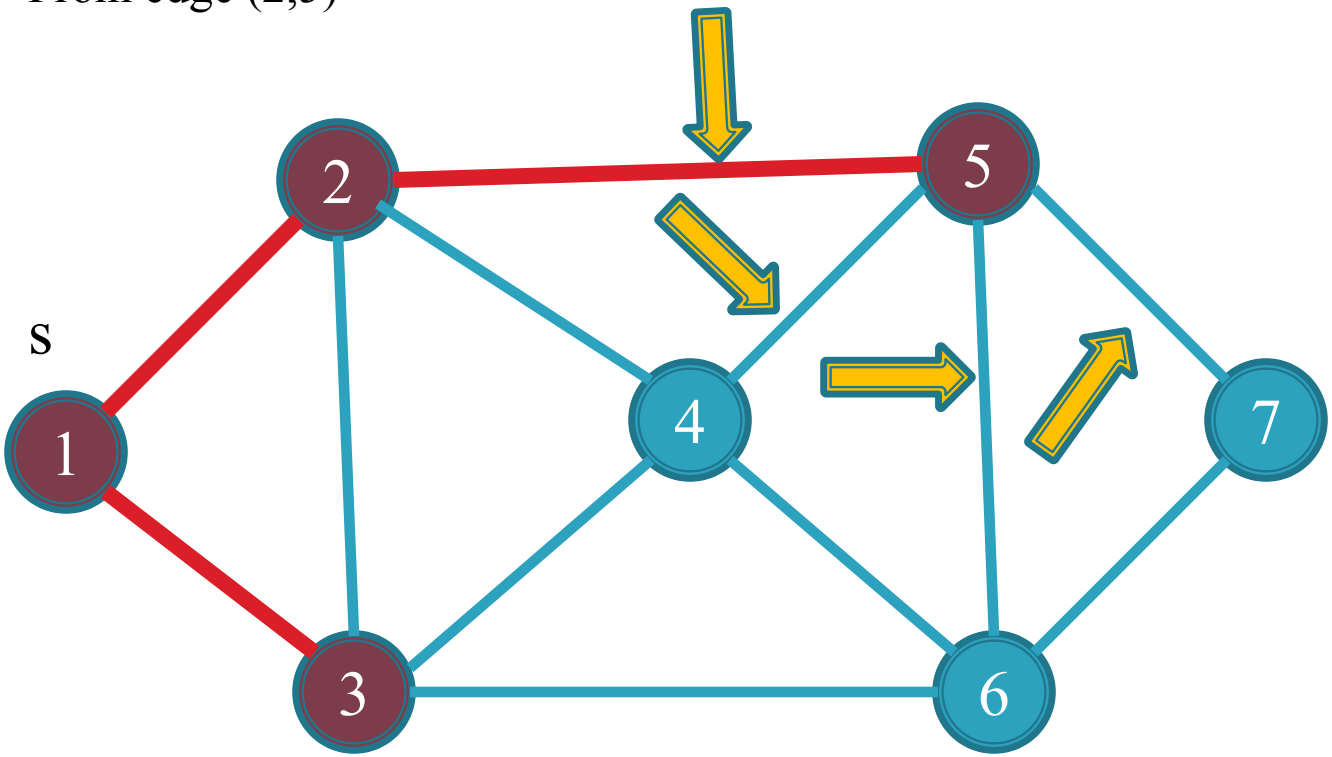

d[3]=d[1]+1=0+1=1

| 0 | 1 | 1 | ∞ | ∞ | ∞ | ∞ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

3 , 6
3 , 4
3 , 2
3 , 1
2 , 1
2 , 4
2 , 3
2 , 5

*Queue*

From edge (2,5)



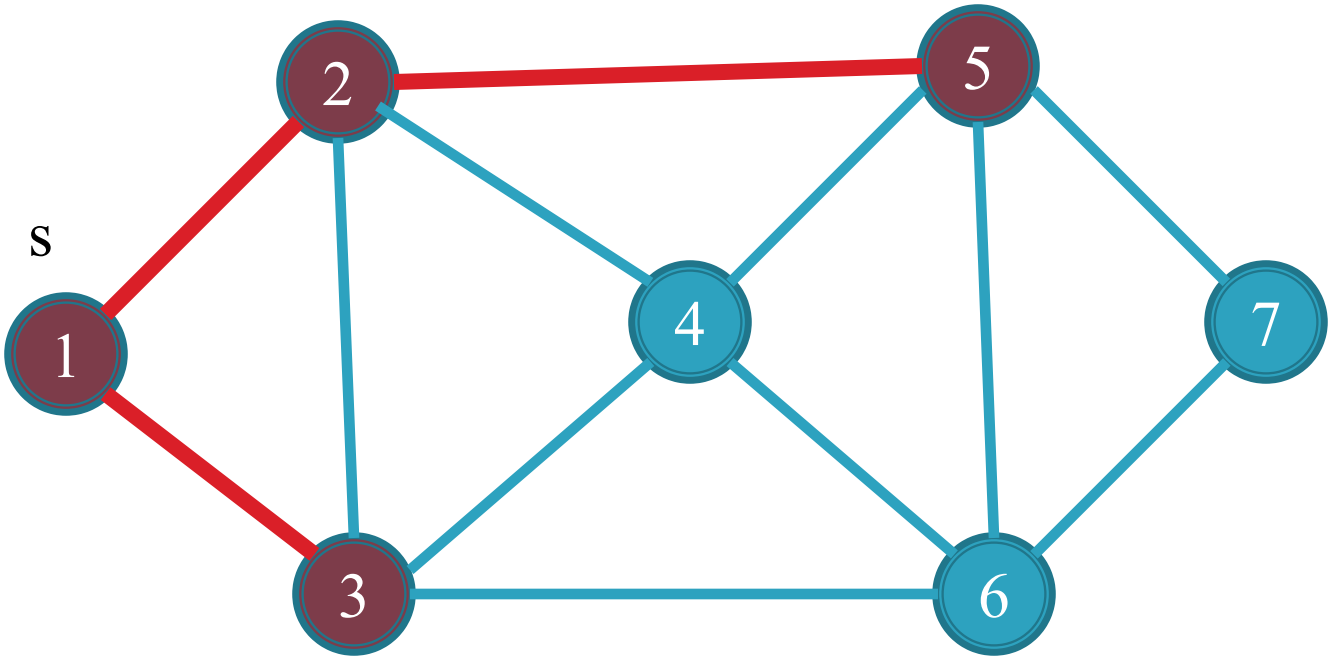

d[5]=d[2]+1=1+1=2

| 0 | 1 | 1 | ∞ | 2 | ∞ | ∞ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

5 , 4
5 , 6
5 , 2
5 , 7
3 , 6
3 , 4
3 , 2
3 , 1
2 , 1
2 , 4
2 , 3

*Queue*

From edge (2,3) the vertex 3 is marked



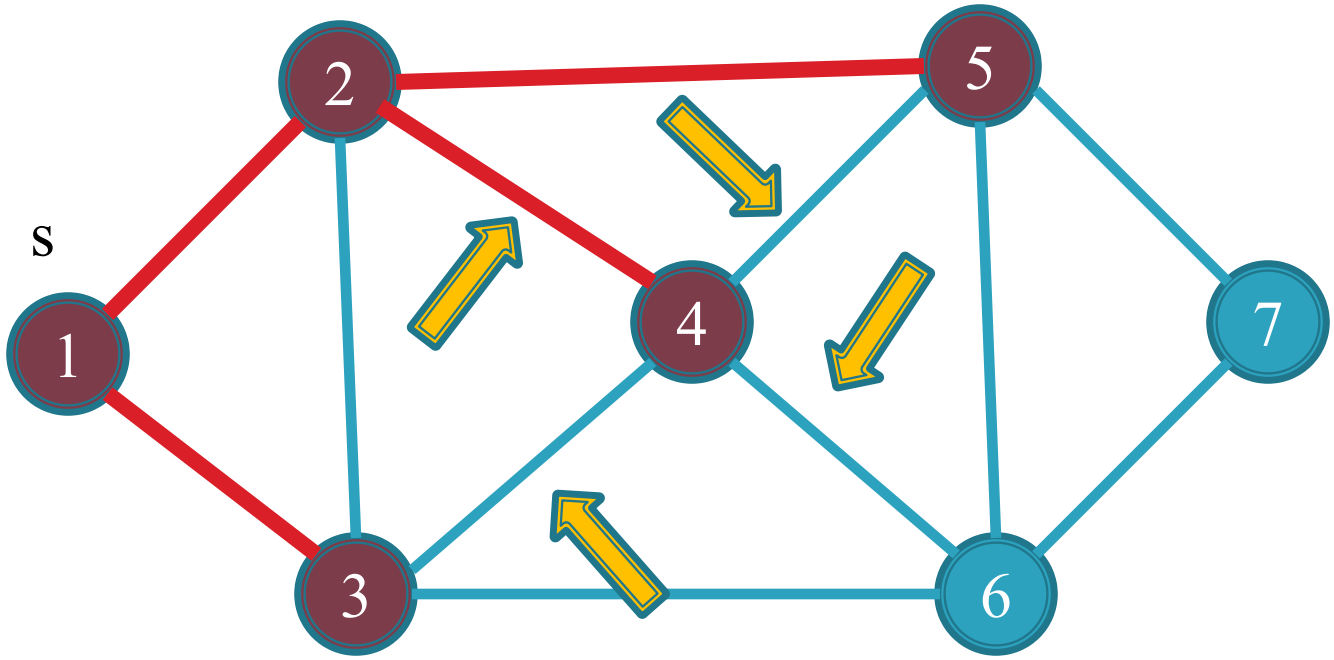

d[5]=d[2]+1=1+1=2

| 0 | 1 | 1 | ∞ | 2 | ∞ | ∞ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

Queue

5 , 4
5 , 6
5 , 2
5 , 7
3 , 6
3 , 4
3 , 2
3 , 1
2 , 1
2 , 4

From edge (2,4)



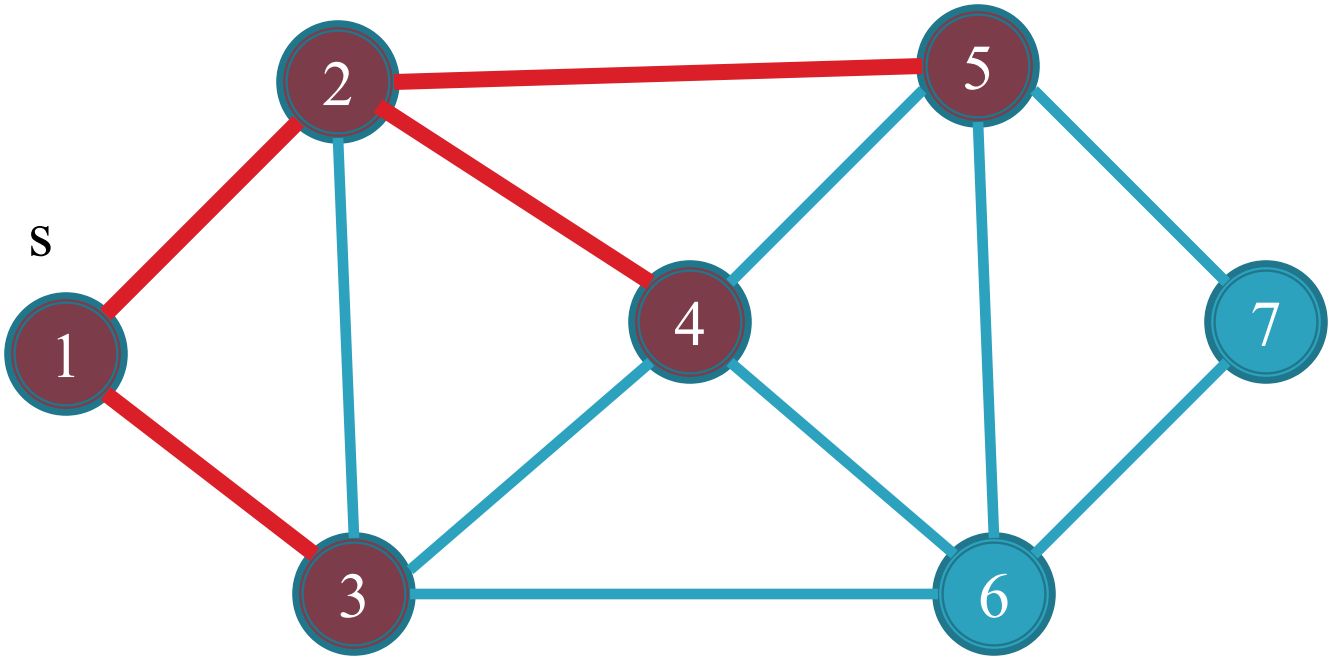

d[4]=d[2]+1=1+1=2

| 0 | 1 | 1 | 2 | 2 | ∞ | ∞ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

4 , 2
4 , 6
4 , 5
4 , 3
5 , 4
5 , 6
5 , 2
5 , 7
3 , 6
3 , 4
3 , 2
3 , 1
2 , 1

*Queue*

From edge (2,1) vertex 1 is marked
From edge (3,1) vertex 1 is marked
From edge (3,2) vertex 2 is marked
From edge (3,4) vertex 4 is marked

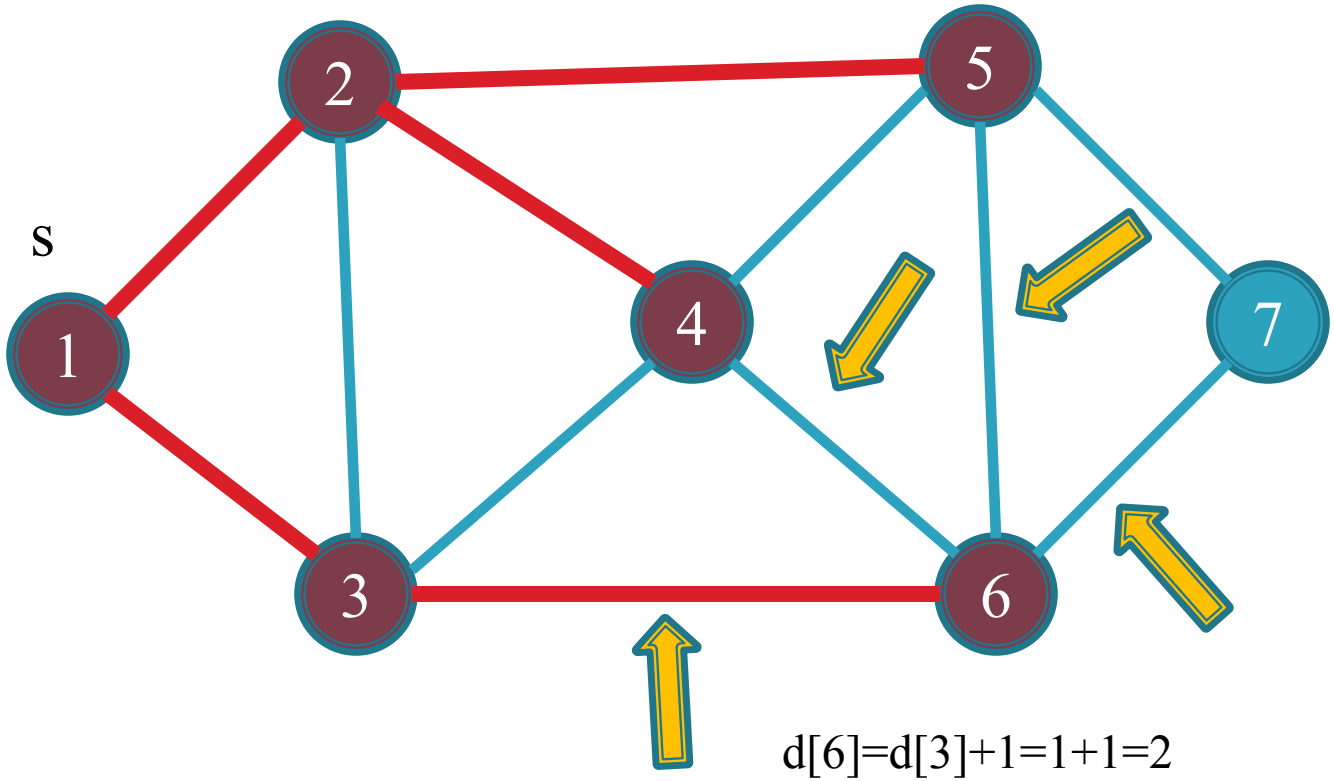



| 0 | 1 | 1 | 2 | 2 | ∞ | ∞ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

4 , 2
4 , 6
4 , 5
4 , 3
5 , 4
5 , 6
5 , 2
5 , 7
3 , 6

*Queue*

From edge (3,6)



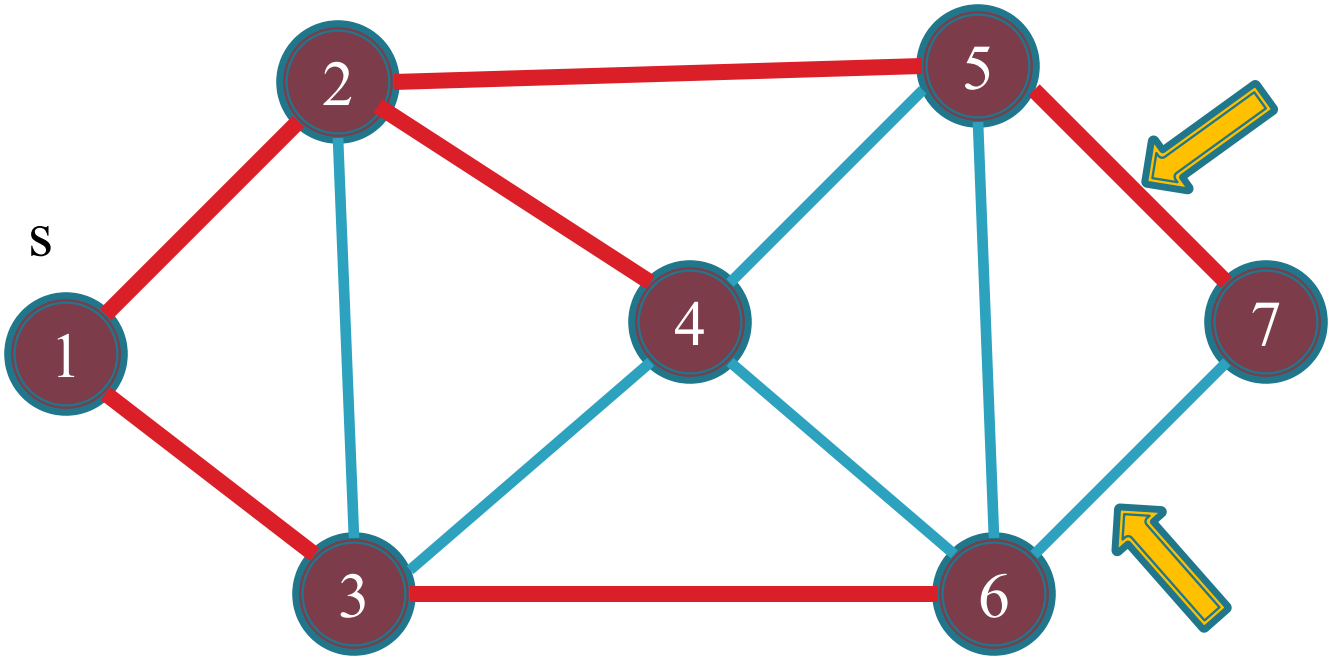

d[6]=d[3]+1=1+1=2

| 0 | 1 | 1 | 2 | 2 | 2 | ∞ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

**Queue**

6 , 7
6 , 4
6 , 5
6 , 3
4 , 2
4 , 6
4 , 5
4 , 3
5 , 4
5 , 6
5 , 2
5 , 7

From edge (5,7)



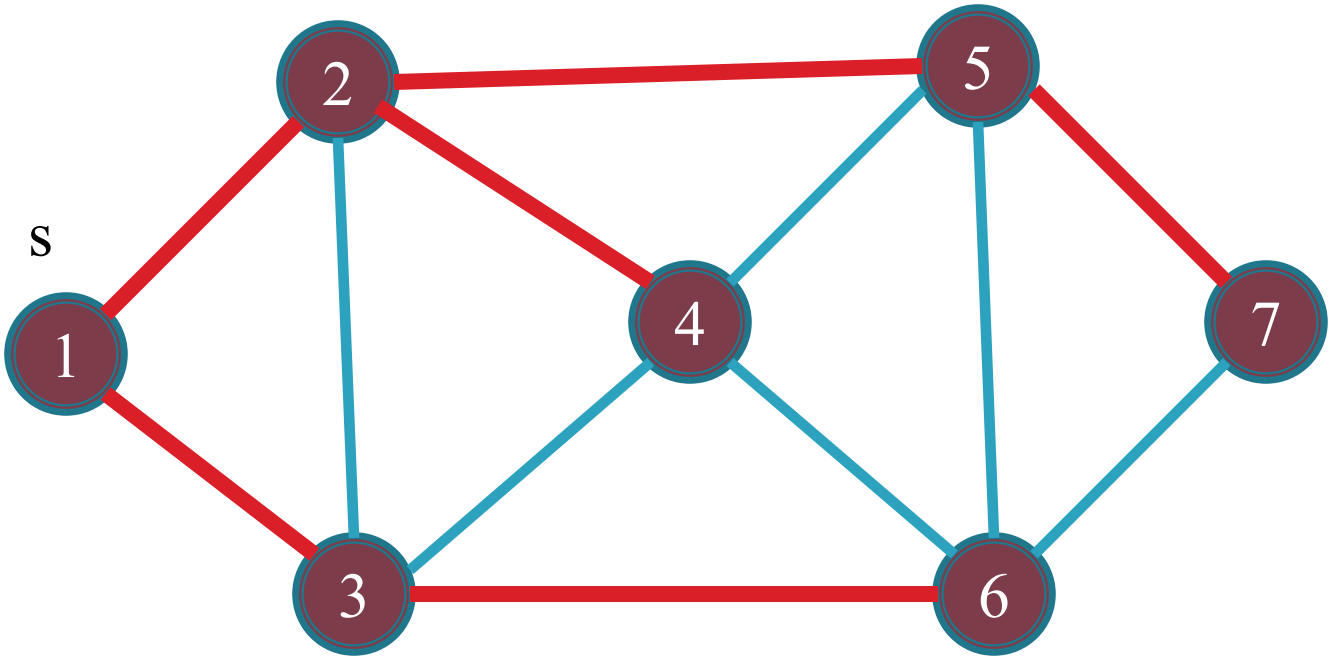

d[7]=d[5]+1=2+1=3

| 0 | 1 | 1 | 2 | 2 | 2 | 3 |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

Queue

7 , 5
7 , 6
6 , 7
6 , 4
6 , 5
6 , 3
4 , 2
4 , 6
4 , 5
4 , 3
5 , 4
5 , 6
5 , 2

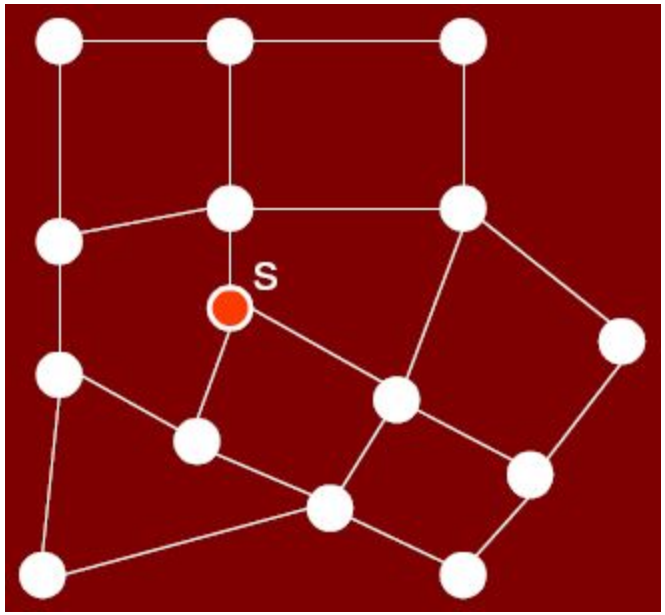
All vertex are marked now queue will become empty




| 0 | 1 | 1 | 2 | 2 | 2 | 3 |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

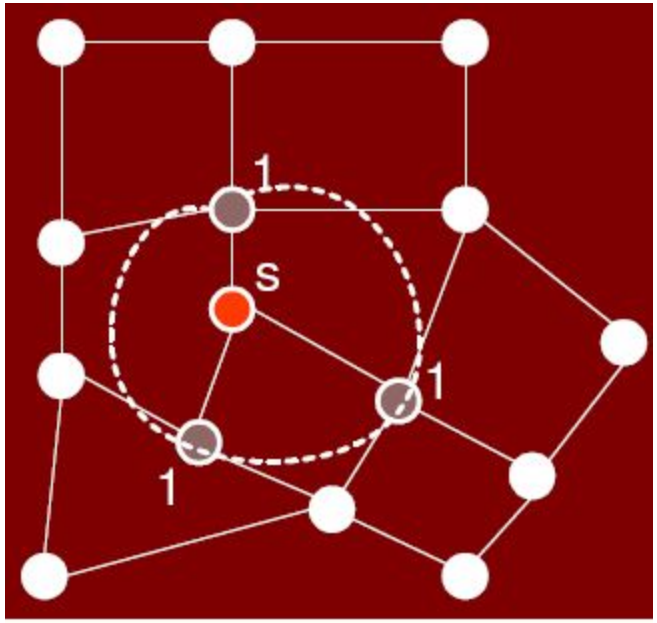
# Shortest path with BFS

- Run the given code on this example.
- Assigned the name of these vertices and solved it.
- How BFS traverse in waveform is shown in this example.
- However a solution is given in next slide.



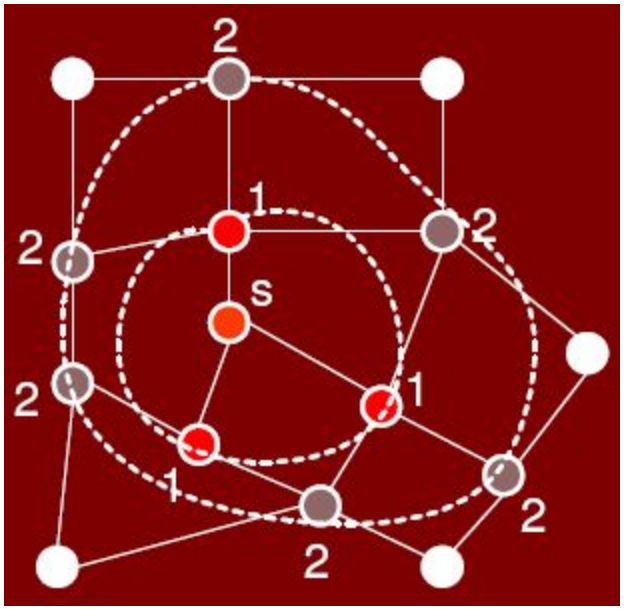
# Wave reaching distance 1 vertices during BFS

All the marked nodes are at distance 1 from source node.



# Wave reaching distance 2 vertices during BFS

All the marked nodes with gray color are at distance 2 from source node.



# Wave reaching distance 3 vertices during BFS

All the marked nodes with gray color are at distance 3 from source node.